

國立臺北商業大學

資訊管理系

114，資訊系統專案設計

系統手冊



組 別：第 114407 組

題 目：MEI 食不打烊

指導老師：葉明貴系主任

組 長：11146061 郭宥妍

組 員：11146066 簡孝宇 11146073 張楷偉

11146083 林勝威

中 華 民 國 114 年 12 月 31 日

目錄

第 1 章	前言	1
1-1	背景介紹	1
1-2	動機	1
1-3	系統目的與目標	2
1-4	預期成果	2
第 2 章	營運計畫	3
2-1	可行性分析	3
2-2	商業模式－Business model	4
2-3	市場分析－STP	5
2-4	競爭力分析 SWOT-TOWS	7
第 3 章	系統規格	9
3-1	系統架構	9
3-2	系統軟、硬體需求與技術平台	11
3-3	使用標準與工具	11
第 4 章	專案時程與組織分工	12
4-1	專案時程	12
4-2	專案組織與分工	13
4-3	GitHub 紀錄	15
第 5 章	需求模型	16
5-1	使用者需求	16
5-2	使用個案圖(Use case diagram)	17
5-3	使用個案描述	17
5-4	分析類別圖(Analysis class diagram)	20
第 6 章	設計模型	21
6-1	循序圖(Sequential diagram)	21
6-2	設計類別圖(Design class diagram)	23
第 7 章	實作模型	24
7-1	佈署圖(Deployment diagram)	24
7-2	套件圖(Package diagram)	24
7-3	元件圖(Component diagram)	26

7-4 狀態機(State machine).....	27
第 8 章 資料庫設計	31
8-1 資料庫關聯表	31
8-2 表格及其 Meta data.....	32
第 9 章 程式	38
9-1 元件清單及其規格描述.....	38
9-2 其他附屬之各種元件	42
第 10 章 測試模型	69
10-1 測試計畫	69
10-2 測試個案與測試結果.....	70
第 11 章 操作手冊	74
11-1 系統元件	74
第 12 章 使用手冊	75
第 13 章 感想	82
第 14 章 參考資料	84
附錄	85

圖目錄

圖 3-1-1：系統架構圖	9
圖 3-1-2：系統流程圖	10
圖 4-3-1：上傳 GitHub 紀錄－11146061 郭宥妍	15
圖 4-3-2：上傳 GitHub 紀錄－11146066 簡孝宇	15
圖 4-3-3：上傳 GitHub 紀錄－11146073 張楷偉	15
圖 4-3-4：上傳 GitHub 紀錄－11146083 林勝威	15
圖 5-2-1：使用個案圖	17
圖 5-4-1：分析類別圖	20
圖 6-1-1：循序圖－登入/註冊	21
圖 6-1-2：循序圖－餐廳查詢	21
圖 6-1-3：循序圖－餐廳評論	22
圖 6-1-4：循序圖－捷運查詢	22
圖 6-2-1：設計類別圖	23
圖 7-1-1：佈署圖	24
圖 7-2-1：套件圖－登入/註冊模組	24
圖 7-2-2：套件圖－餐廳管理模組	25
圖 7-2-3：套件圖－評論模組	25
圖 7-2-4：套件圖－捷運/站點模組	25
圖 7-2-5：套件圖－用戶管理模組	26
圖 7-2-6：套件圖－首頁/重定向模組	26
圖 7-3-1：元件圖	26
圖 7-4-1：狀態機－登入/註冊流程	27
圖 7-4-2：狀態機－用戶（帳號啟用/停權/管理員）	28
圖 7-4-3：狀態機－餐廳營業	28
圖 7-4-4：狀態機－評論發佈流程	29
圖 7-4-5：狀態機－前端頁面/表單切換	29
圖 7-4-6：狀態機－首頁重定向	30
圖 8-1-1：MySQL 資料庫 ER 圖	31
圖 8-1-2：資料庫關聯圖	32
圖 11-1-1：註冊頁面	75

圖 11-1-2：登入頁面.....	75
圖 11-1-3：首頁頁面.....	76
圖 11-1-4：站點查詢.....	76
圖 11-1-5：熱門站即時資訊	77
圖 11-1-6：當月 TOP 10 店家	77
圖 11-1-7：夜貓子專區.....	78
圖 11-1-7：喝酒專區.....	78
圖 11-1-9：打卡任務.....	79
圖 11-1-10：美食地圖.....	80
圖 11-1-11：收藏及通知.....	80
圖 11-1-12：會員中心.....	81
圖 11-1-13：收藏美食.....	81

表目錄

表 2-3-1：潛在使用者區分為三大族群	5
表 2-3-2：平台主打特色	6
表 2-4-1：SWOT 分析	7
表 2-4-2：TOWS 分析	8
表 3-2-1：系統軟、硬體需求與技術平台	11
表 3-3-1：系統環境	11
表 3-3-2：系統開發工具	11
表 3-3-3：程式開發技術	11
表 3-3-4：文件編寫工具	11
表 3-3-5：管理程式平台	11
表 4-1-1：專案時程時程表	12
表 4-2-1：專案組織與分工	13
表 4-2-2：工作內容	14
表 5-1-1：使用者需求表	16
表 5-3-1：登入	17
表 5-3-2：註冊	18
表 5-3-3：查詢餐廳	18
表 5-3-4：查看餐廳詳細資訊	19
表 5-3-5：發表評論/打卡	19
表 8-2-1：資料表	32
表 8-2-2：user.....	33
表 8-2-3：metroline	33
表 8-2-4：station.....	34
表 8-2-5：restaurant.....	34
表 8-2-6：businesshours	35
表 8-2-7：review.....	36
表 8-2-8：checkinreview	36
表 8-2-9：traininfo.....	37
表 8-2-10：stationid.....	37
表 9-1-1：元件清單與規格描述表(前端)－網頁介面撰寫	38

表 9-1-2：元件清單與規格描述表(前端)－網頁介面設計	38
表 9-1-3：元件清單與規格描述表(前端)－網頁動畫設計	39
表 9-1-4：元件清單與規格描述表(後端)－爬蟲	40
表 9-1-5：元件清單與規格描述表(後端)－讀取 API.....	40
表 9-1-6：元件清單與規格描述表(後端)－Service	41
表 9-1-7：元件清單與規格描述表(後端)－定時更新資訊	41
表 9-1-8：元件清單與規格描述表(後端)－使用者資訊管理	41
表 9-1-9：元件清單與規格描述表(後端)－串接相關網頁	41
表 9-2-1：虛擬碼－homepage.html、homepage.css	42
表 9-2-2：虛擬碼－Member_center.html、Member_center.css.....	44
表 9-2-3：虛擬碼－night.html、night.css.....	46
表 9-2-4：虛擬碼－mission.js	49
表 9-2-5：虛擬碼－stationdata.js	50
表 9-2-6：虛擬碼－metro_train_service.py	52
表 9-2-7：虛擬碼－models.py.....	55
表 9-2-8：虛擬碼－views.py	57
表 9-2-9：虛擬碼－serializers.py	62
表 9-2-10：虛擬碼－review_service.py	65
表 10-2-1：測試結果－會員註冊	70
表 10-2-2：測試結果－會員登入	70
表 10-2-3：測試結果－更改會員資料	70
表 10-2-4：測試結果－站點查詢	71
表 10-2-5：測試結果－當月 TOP 10 店家.....	71
表 10-2-6：測試結果－夜貓子專區	71
表 10-2-7：測試結果－喝酒專區	72
表 10-2-8：測試結果－打卡任務評論	72
表 10-2-9：測試結果－美食任務版	72
表 10-2-10：測試結果－捷運時刻表	73
表 10-2-11：測試結果－美食地圖	73
表 10-2-12：測試結果－收藏及通知	73

第 1 章 前言

1-1 背景介紹

台北捷運系統作為都市運輸的重要骨幹，每日承載大量通勤與觀光人潮，其沿線站點涵蓋多個商圈與飲食聚落，具有高度的區域特色與美食潛力。而現有的美食資訊平台多數以行政區域或餐廳類型分類為主，缺乏以「捷運站」為導向的查詢邏輯，對依賴大眾運輸移動的使用者而言，難以快速掌握特定站點周邊的餐飲資訊。

其次，隨著社群媒體風潮興起，越來越多使用者習慣透過「打卡」、「拍照上傳」等方式分享美食與生活體驗，社群互動逐漸成為資訊擴散的重要來源。導入會員制度與互動任務機制，已成為提升平台活躍度與使用者參與的主流設計方向。

此外，夜間活動人口對於深夜餐飲選擇與交通回程規劃仍面臨資訊不足的問題，顯示夜間生活相關資訊整合的需求仍有成長空間。

1-2 動機

隨著智慧型手機與網路服務的快速發展，民眾在外出時越來越依賴行動裝置來查詢即時資訊，特別是在交通與美食領域的需求尤為顯著。台北捷運系統作為首都地區最重要的公共運輸網絡，每日串聯數百萬人次的通勤與觀光流動，其各站周邊聚集了大量在地美食資源。然而，目前市面上仍缺乏一個以「捷運站」為核心，整合交通、美食、社群服務的互動平台。

1-3 系統目的與目標

本系統旨在打造一個以台北捷運站為主軸的美食資訊平台，使用者可透過手機查詢各站周邊的推薦餐廳與深夜美食資訊。平台採用捷運站導向的查詢方式，提升資訊搜尋效率，並結合會員登入、拍照打卡、評論與任務等互動功能，強化社群參與與內容豐富度。此外，設置「夜貓子專區」，整合深夜營業店家與捷運末五班車時刻，協助夜間用戶安排行程。系統也導入推播功能，主動提醒捷運列車即將到站的時間，提升使用者的行動便利性。整體平台支援繁體中文語言，並採用響應式設計，確保行動裝置上的良好操作體驗，打造一個實用、友善且具互動性的美食導覽平台。

1-4 預期成果

我們預期系統可以以獨家設置「夜貓子專區」，整合深夜營業店家與末班車時刻，專為夜間行動者量身打造，貼心守護每一個夜晚的旅程。平台支援繁體中文與響應式設計，無論身處何地都能輕鬆操作，串聯每位饕客與捷運沿線美味。讓行動導覽與社群美食兼具，成就智慧城市的新型態餐飲探索體驗，進而達到「美味同行，捷運每一站都精彩！」。

第 2 章 營運計畫

2-1 可行性分析

● 技術可行性

1. 本系統採用成熟穩定的 Web 技術，包括 HTML5、CSS、JavaScript、Fetch API 資料處理與響應式設計，並搭配 Django 等後端框架進行資料串接與展示。
2. 使用者功能包含捷運站導向的美食查詢、夜貓子餐廳推薦、打卡評論與多語操作，皆為可實作功能，已分工執行中。

● 資料可行性

1. 我們實際向台北捷運官方申請了站點資料 API，取得各站相關資訊與即時列車資訊作為查詢基礎。
2. 同時也向 Google Places API 正式申請金鑰，用以補足店家圖片、營業資訊、使用者評論等資料，確保內容即時且具參考價值。
3. 除了 API 資料，也可搭配必要的手動輸入測試資料與模擬使用者評論來建立測試案例與展示效果。

● 操作可行性

1. 系統導向捷運站點搜尋邏輯，搭配夜間資訊與推播提醒，符合真實情境需求。
2. 支援繁體中文語言，提供觀光友善環境，並優化手機使用體驗，易於推廣與實際使用。

- 實施可行性

1. 已完成 API 串接測試與資料顯示範例，分工包含前端介面設計、後端串接邏輯、資料處理與互動功能開發，具備落地實作能力。
2. 系統開發過程將持續依照專題進度規劃逐步整合並測試。

2-2 商業模式－Business model

- 關鍵夥伴（Key Partners）

1. 台北捷運公司（提供站點資訊 API）
2. Google Places API（提供餐廳詳細資訊與圖片）
3. 開放資料平台（補足區域與地點資訊）

- 關鍵活動（Key Activities）

1. 建立捷運站導向的美食搜尋系統
2. 串接捷運與 Google API 資料
3. 設計互動功能（打卡、評論、任務）
4. 提供夜間美食與交通資訊整合
5. 響應式網站開發

- 價值主張（Value Proposition）

1. 台北第一個「以捷運站為中心」的美食與夜貓資訊平台
2. 解決夜間餐飲選擇難
3. 支援繁體中文與行動操作，觀光友善、通勤方便
4. 提供「打卡任務」、「夜間專區」、等創新互動功能

2-3 市場分析－STP

● 市場區隔 Segmentation

依據使用者的行為特性與使用情境，可將潛在使用者區分為下列三大族群：

▼表 2-3-1：潛在使用者區分為三大族群

區隔族群	特徵說明
通勤族群	以捷運作為每日上下班交通工具，對「午餐、晚餐」推薦有高度需求。使用時間集中於早上與傍晚。
夜間活動族	活動時間為夜間，常有深夜餐飲需求（如學生、夜班族、夜生活族群）。關心餐廳營業時間與回家交通。
國內自由行旅客	在台北捷運沿線移動，對地點不熟，習慣使用手機查詢。常有語言障礙，關心推薦、地圖、評論支援。

● 目標對象 Targeting

平台的首波目標使用者為：

1. 以台北捷運為主要移動方式的通勤者與觀光客

☞ 因平台以「捷運站」為搜尋核心，特別適合此類使用者快速找到附近美食。

2. 對夜間生活有需求的年輕族群或上夜班者

☞ 夜間外食資訊缺乏，平台設置「夜貓子專區」與「末班車時刻」剛好填補空白。

● 產品定位 Positioning

「全台第一個以捷運站為導向的多語美食地圖，深夜行動族群的最佳夥伴」

▼表 2-3-2：平台主打特色

定位主軸	說明
捷運站導向	使用者以車站為起點查找周邊美食，符合實際移動邏輯。
夜貓子專區	整合深夜營業餐廳與末班車資訊，貼近夜間需求。
社群互動	提供拍照打卡、評論任務，提升參與感與內容豐富度。

2-4 競爭力分析 SWOT-TOWS

▼表 2-4-1：SWOT 分析

優勢 Strengths	劣勢 Weaknesses
➡ 以捷運站為導向，查詢邏輯清晰直覺 ➡ 結合「夜間營業」與「班車提醒」，功能創新 ➡ 提供打卡、評論、任務互動功能，增加使用體驗 ➡ 系統介面採響應式設計，適用各種裝置操作	➡ 初期資料仍須人工補充，規模有限 ➡ 評論與使用者互動資料需時間累積 ➡ 需仰賴外部 API（捷運與 Google）穩定度 ➡ 系統需持續維護與更新，開發期工作量大
機會 Opportunities	威脅 Threats
➡ 台北捷運系統龐大，站點周邊美食豐富 ➡ 使用者對於夜間資訊與在地推薦有實際需求 ➡ 行動裝置使用普及，適合推動即時查詢功能 ➡ 開放資料與公共 API 提供資料串接支援	➡ 類似平台（如 Google Maps、美食社群 App）功能重疊 ➡ 多語內容需人工處理，語言品質需控管 ➡ API 數據可能改版或停止提供，存在依賴風險 ➡ 資料更新頻率與正確性會影響使用者信任 ➡ 多語內容維護成本高，翻譯準確度需控制

▼表 2-4-2：TOWS 分析

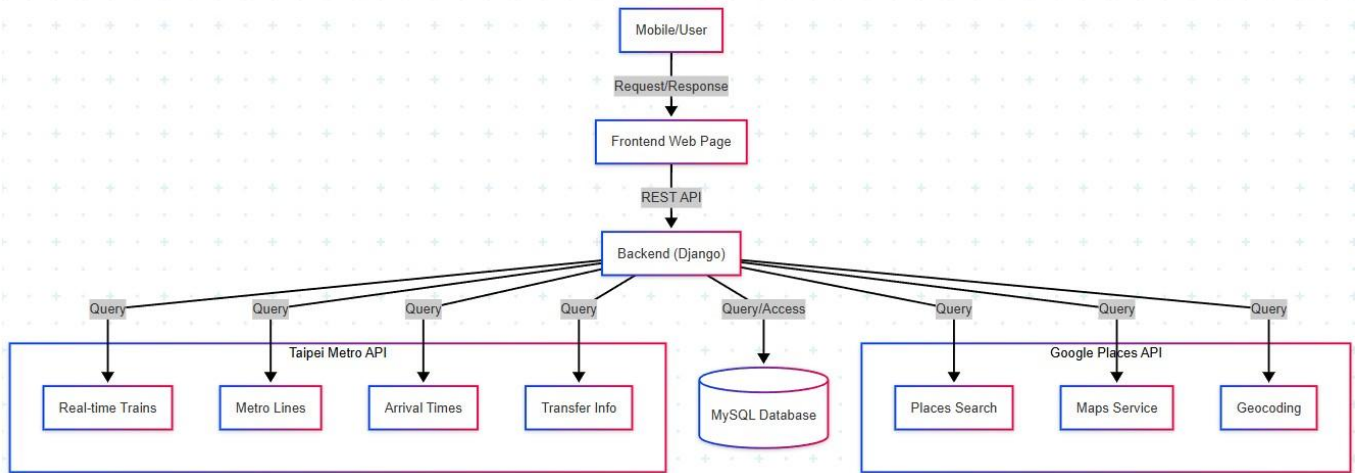
內部因素 外在因素	優勢 Strengths	劣勢 Weaknesses
機會 Opportunities	SO 策略（增長性策略） ➡ 利用捷運導向與夜間資訊整合特色，解決使用者移動與用餐交會問題 ➡ 運用中文介面設計，提升對國內觀光族的實用性與友善度 ➡ 採模組化架構，未來可擴展至其他縣市捷運系統	WO 策略（移轉性策略） ➡ 初期聚焦重點站點，逐步擴充內容並建立測試資料 ➡ 結合 API 與手動輸入方式補足資料來源，提高資料覆蓋率 ➡ 強化錯誤回報與使用回饋機制，逐步優化功能
威脅 Threats	ST 策略（多角化策略） ➡ 強調系統特色（捷運導向＋班車提醒＋夜間美食）與現有平台做出區隔 ➡ 建立 API 備援處理流程，減少依賴性造成的風險 ➡ 設計簡潔、操作直觀，提升易用性以留住使用者	WT 策略（防禦型策略） ➡ 提供基礎版本測試範圍，確保資料正確與功能完整後再逐步擴展 ➡ 強化多語翻譯管理，方便維護 ➡ 規劃資料更新流程與開發紀錄，方便未來交接或維護

綜合 SWOT 與 TOWS 分析可見，本系統具備以「捷運站」為導向的查詢優勢，搭配夜間美食資訊、末班車時刻支援，成功區隔於現有以地區或餐廳類型為主的美食平台，展現明確的功能特色與實用價值。

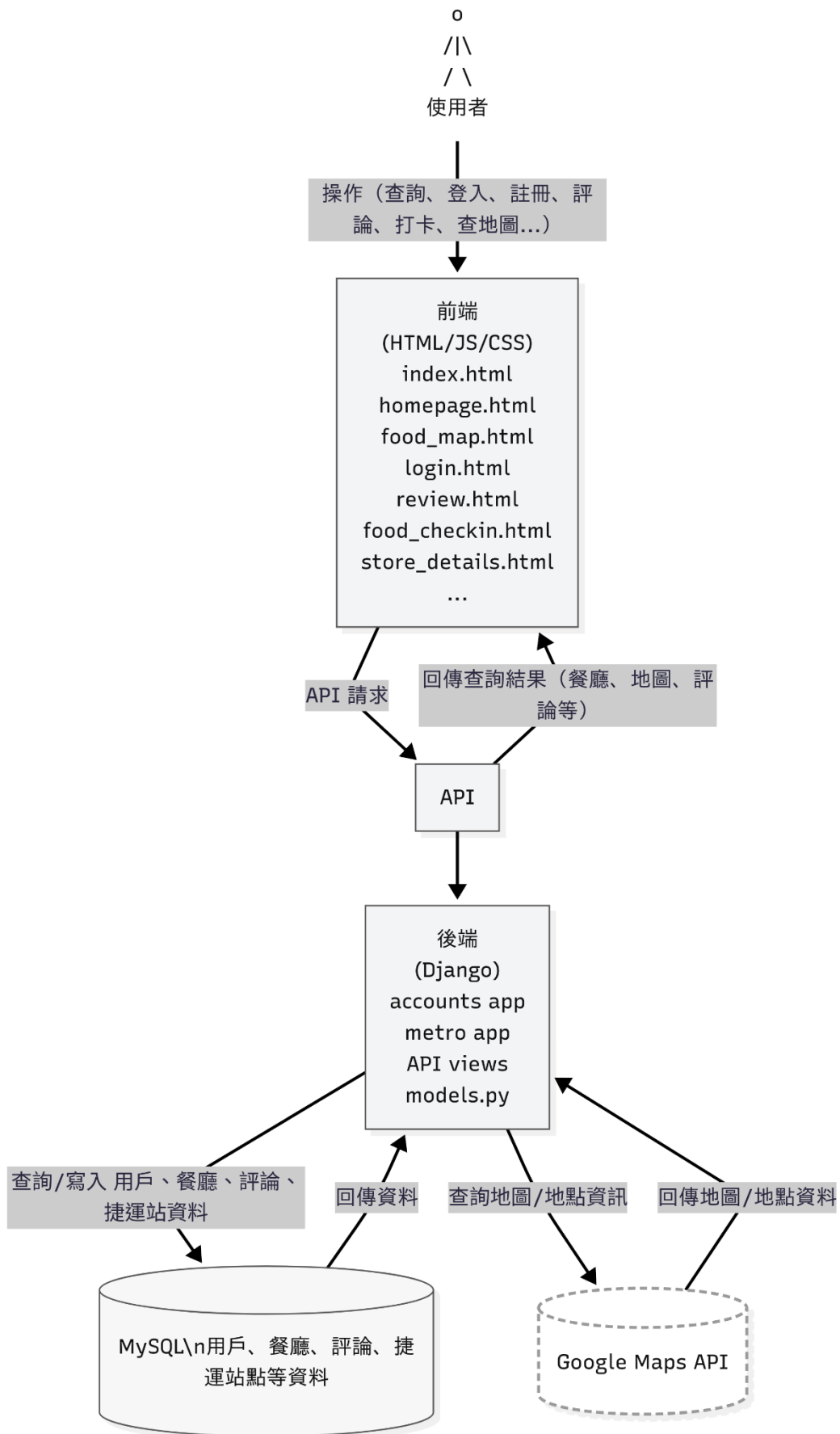
然而，系統初期仍面臨資料覆蓋率不足、API 依賴度高與語言維護成本等挑戰，須透過階段性擴充、錯誤回報機制與多語翻譯策略逐步克服。透過模組化設計與漸進式開發，可有效提升平台穩定性與可維護性，並為未來推展至其他縣市或新增功能保留彈性。

第 3 章 系統規格

3-1 系統架構



▲圖 3-1-1：系統架構圖



▲圖 3-1-2：系統流程圖

3-2 系統軟、硬體需求與技術平台

▼表 3-2-1：系統軟、硬體需求與技術平台

設備	規格
手機裝置	
作業系統	iOS、Android
版本	iOS 全系列、Android 8 以上
操作介面	Safari、Chrome

3-3 使用標準與工具

▼表 3-3-1：系統環境

作業系統	Windows 11
網路需求	WiFi/有線網路

▼表 3-3-2：系統開發工具

網頁設計	Visual Studio Code
資料庫	MySQL
伺服器工具	Django

▼表 3-3-3：程式開發技術

前端技術	HTML、CSS、JavaScript
後端技術	Python、Django
API 串接	RESTful API、Google Places API、台北捷運 API

▼表 3-3-4：文件編寫工具

繪圖工具	Canva、Flaticon
簡報製作	Microsoft PowerPoint 2016
文件製作	Microsoft Word 2021

▼表 3-3-5：管理程式平台

專案管理	github、fork
------	-------------

第 4 章 專案時程與組織分工

4-1 專案時程

▼表 4-1-1：專案時程時程表

	預期進度												實際進度											
	1 月	2 月	3 月	4 月	5 月	6 月	7 月	8 月	9 月	10 月	11 月	12 月	1 月	2 月	3 月	4 月	5 月	6 月	7 月	8 月	9 月	10 月	11 月	12 月
主題構思																								
相關資料蒐集																								
系統功能分析																								
系統模型																								
UI/UX 設計																								
Logo 設計																								
Web 開發																								
資料庫設計																								
資料庫建立																								
API 設定																								
後端開發																								
系統測試																								
系統整合																								
初評系統手冊																								
複評系統手冊																								
系統簡報																								

4-2 專案組織與分工

▼表 4-2-1：專案組織與分工

●主要負責人 ○次要負責人

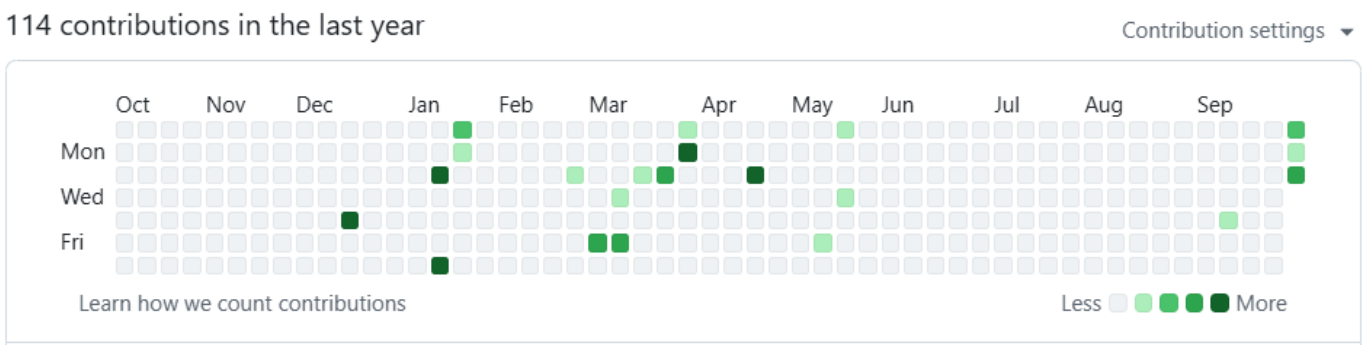
組 員		11146061	11146066	11146073	11146083
項 目		郭宥妍	簡孝宇	張楷偉	林勝威
後端 開發	資料庫建置				●
	伺服器架設				●
	API 設定				●
	捷運資料轉換				●
前端 開發	登入/註冊	○		○	●
	首頁	○	○	●	
	夜貓子專區	○	○	●	
	美食打卡	○	○	●	
	捷運時刻表		○	●	
	美食地圖		○	●	
	通知		○	●	
	會員中心		○	●	
	Javascript 程式		●		
美術 設計	UI / UX	●			
	Web 介面設計	●		○	
	色彩設計	●			
	Logo 設計	●			
	素材設計	●			
文件 撰寫	統整	●			
	第 1 章 前言	●			
	第 2 章 營運計畫	○	●		
	第 3 章 系統規格				●
	第 4 章 專題時程與組織分工	●			
	第 5 章 需求模型				●

組 員		11146061	11146066	11146073	11146083
項 目		郭宥妍	簡孝宇	張楷偉	林勝威
	第 6 章 設計模型				●
	第 7 章 實作模型				●
	第 8 章 資料庫設計				●
	第 9 章 程式	●	○		○
	第 10 章 測試模型	●			
	第 11 章 操作手冊	●			
	第 12 章 使用手冊	●			
報告	簡報製作		●		
影片	系統影片製作	●			

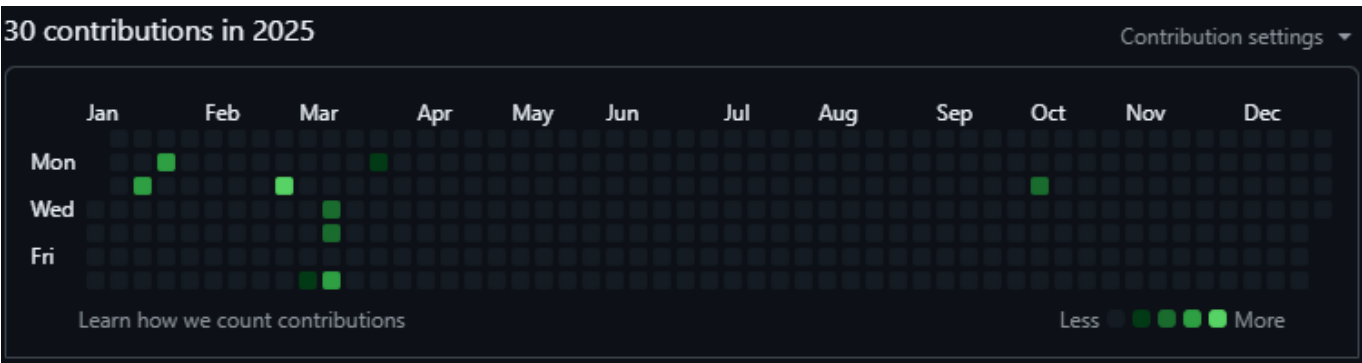
▼表 4-2-2：工作內容

序號	姓名	工作內容	貢獻度
1	組長 <u>郭宥妍</u>	文件撰寫、文件排版、文件統整、Logo 設計、 UI/UX 設計	25%
2	組員 <u>張楷偉</u>	前端網頁設計、前端開發	20%
3	組員 <u>簡孝宇</u>	前端動畫開發、串接 API、簡報製作、文件撰寫	25%
4	組員 <u>林勝威</u>	前後端串接、後端開發、資料庫建置、伺服器架 設、API 設定	30%
			總計：100%

4-3 GitHub 紀錄



▲圖 4-3-1：上傳 GitHub 紀錄－11146061 郭宥妍



▲圖 4-3-2：上傳 GitHub 紀錄－11146066 簡孝宇



▲圖 4-3-3：上傳 GitHub 紀錄－11146073 張楷偉



▲圖 4-3-4：上傳 GitHub 紀錄－11146083 林勝威

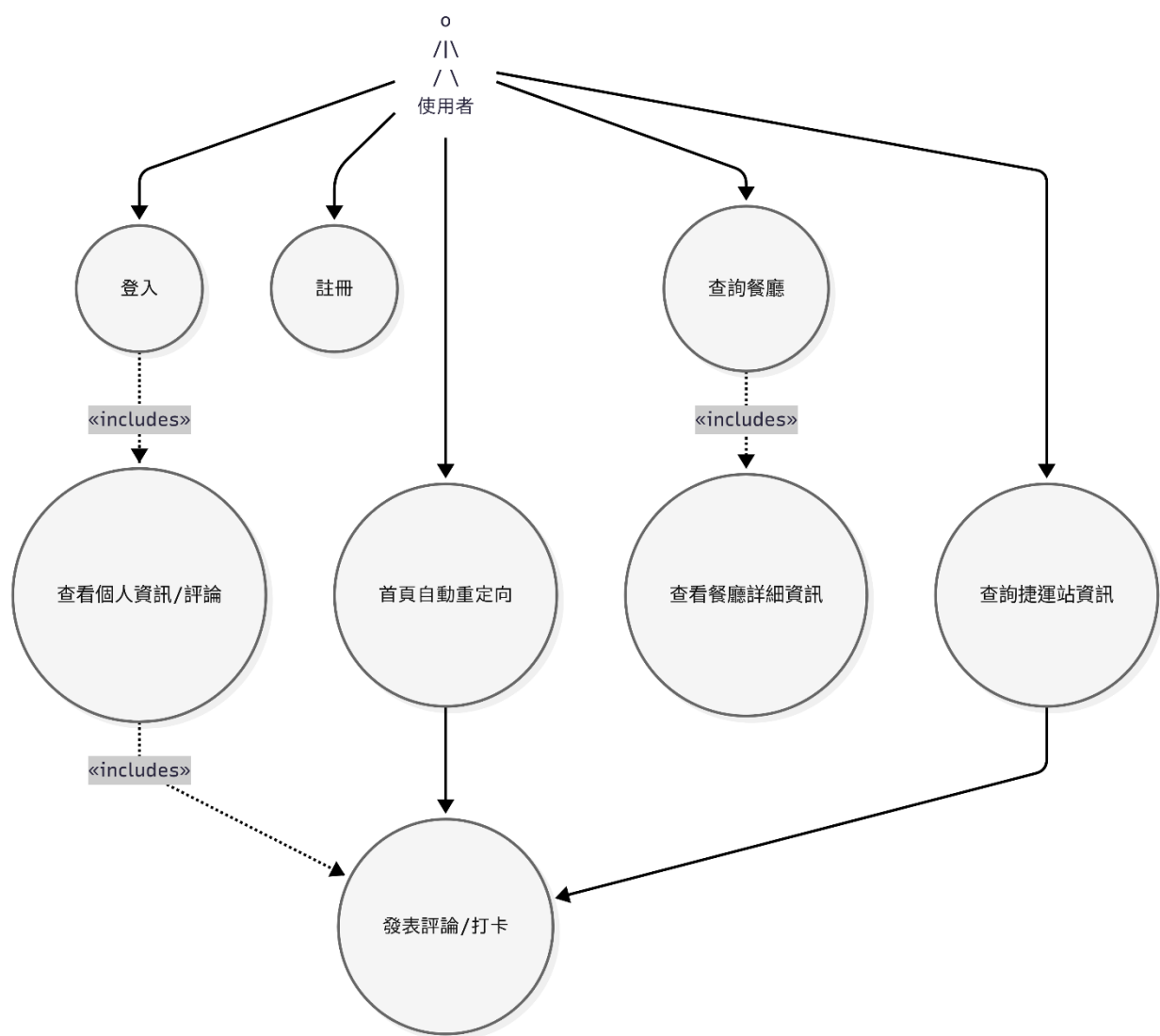
第 5 章 需求模型

5-1 使用者需求

▼表 5-1-1：使用者需求表

UseCase	需求說明
登入與註冊	使用者能夠透過電子郵件註冊帳號並登入系統，進行個人化操作。
餐廳/捷運站查詢	使用者可以透過捷運站、餐廳名稱或地圖查詢附近的餐廳，快速找到想要的美食。
查看餐廳詳細資訊	使用者能夠查看餐廳的詳細資料（名稱、地址、評分、分類、營業時間、評論等）。
發表評論/打卡	使用者可以針對餐廳發表評論、評分，或進行美食打卡，分享用餐體驗。
查詢捷運站資訊	使用者可以查詢捷運站的基本資訊、所屬路線及附近的餐廳。
查看個人資訊/評論	使用者可以在個人頁面查看自己的基本資料、發表過的評論紀錄。
首頁自動重定向	使用者進入首頁時，系統會自動判斷登入狀態並導向對應頁面（已登入導向主頁，未登入導向登入頁）。

5-2 使用個案圖(Use case diagram)



▲圖 5-2-1：使用個案圖

5-3 使用個案描述

▼表 5-3-1：登入

使用個案名稱：登入	
主要參與者	使用者
前置條件	使用者已註冊帳號
事件路徑：	
流程	系統回應
1.使用者輸入信箱與密碼	
2.按下登入	

3.系統驗證帳密	顯示登入成功或失敗訊息
4.登入成功導向首頁	

▼表 5-3-2：註冊

使用個案名稱：註冊	
主要參與者	使用者
前置條件	無（新用戶）
事件路徑：	
流程	系統回應
1.使用者輸入信箱、密碼、確認密碼	
2.按下註冊	
3.系統驗證資料	顯示註冊成功或失敗訊息
4.註冊成功導向登入頁	

▼表 5-3-3：查詢餐廳

使用個案名稱：查詢餐廳	
主要參與者	使用者
前置條件	無
事件路徑：	
流程	系統回應
1.用者於首頁、地圖頁、夜市頁等選擇捷運站或輸入關鍵字	
2.系統顯示符合條件的餐廳列表或地圖標記	
3.點擊餐廳可查看詳細資訊	顯示餐廳列表、地圖標記與詳細資訊

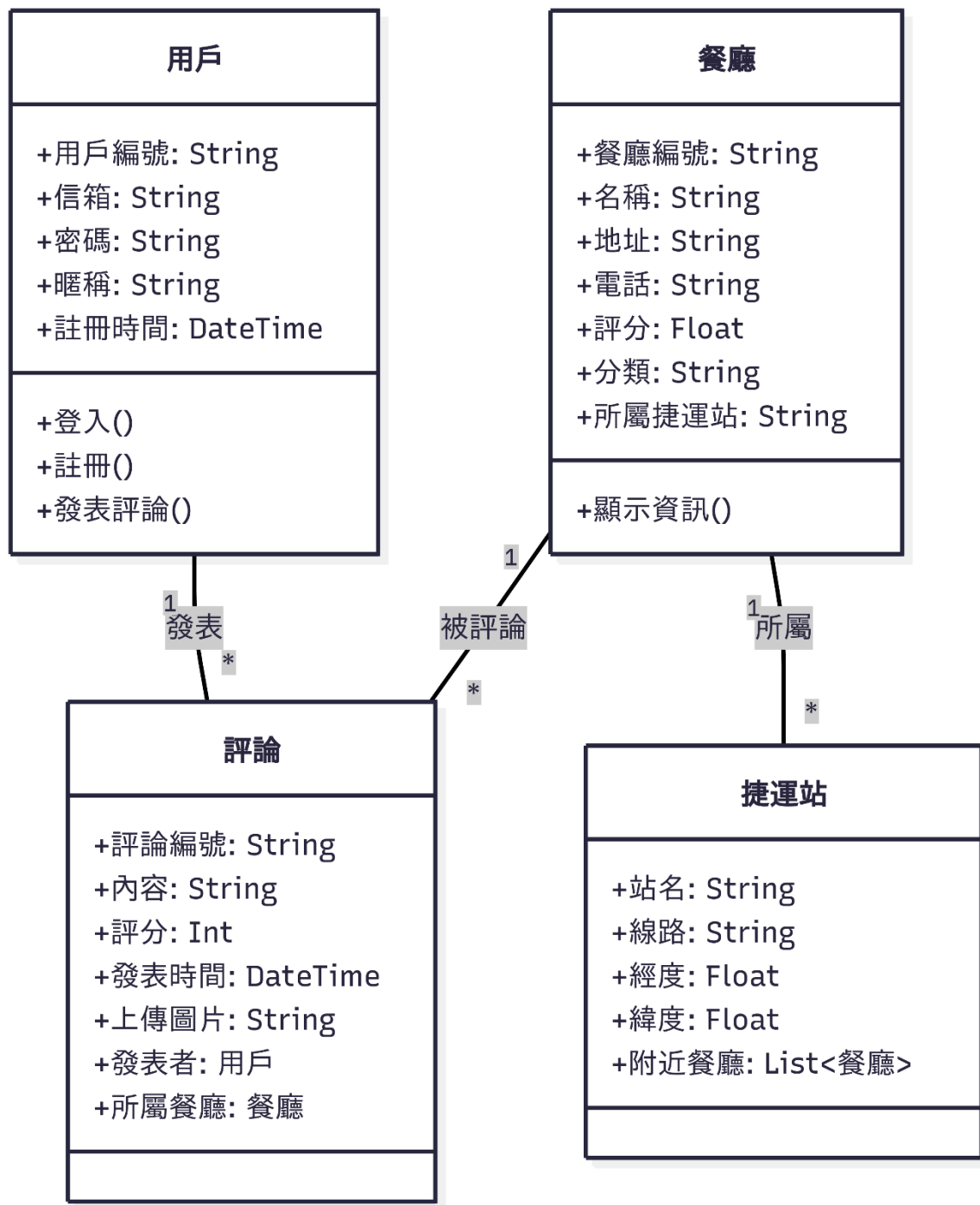
▼表 5-3-4：查看餐廳詳細資訊

使用個案名稱：查看餐廳詳細資訊	
主要參與者	使用者
前置條件	無
事件路徑：	
流程	系統回應
1.使用者於餐廳列表或地圖點擊餐廳	
2.系統顯示餐廳詳細頁	顯示餐廳詳細資訊

▼表 5-3-5：發表評論/打卡

使用個案名稱：發表評論/打卡	
主要參與者	使用者
前置條件	使用者已登入
事件路徑：	
流程	系統回應
1.進入餐廳頁面或打卡頁	
2.輸入評論內容、評分	
3.按下送出	顯示評論/打卡成功或失敗訊息
4.系統儲存評論/打卡資料	

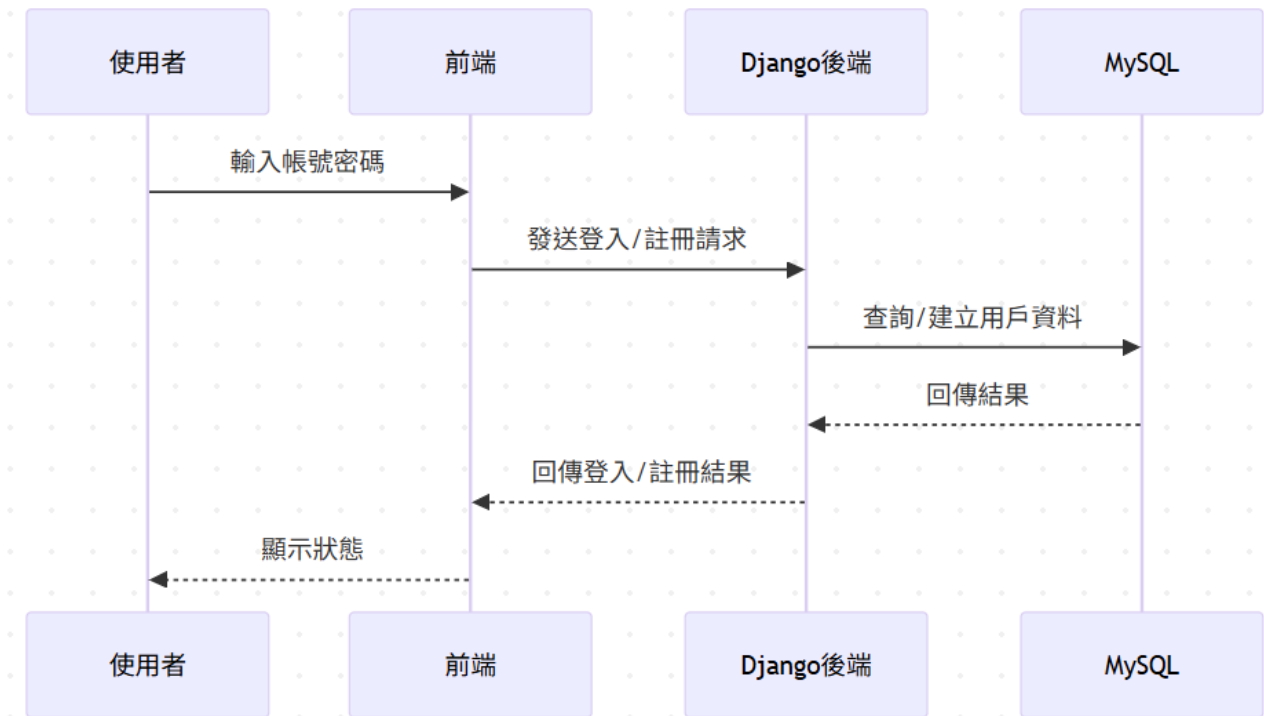
5-4 分析類別圖(Analysis class diagram)



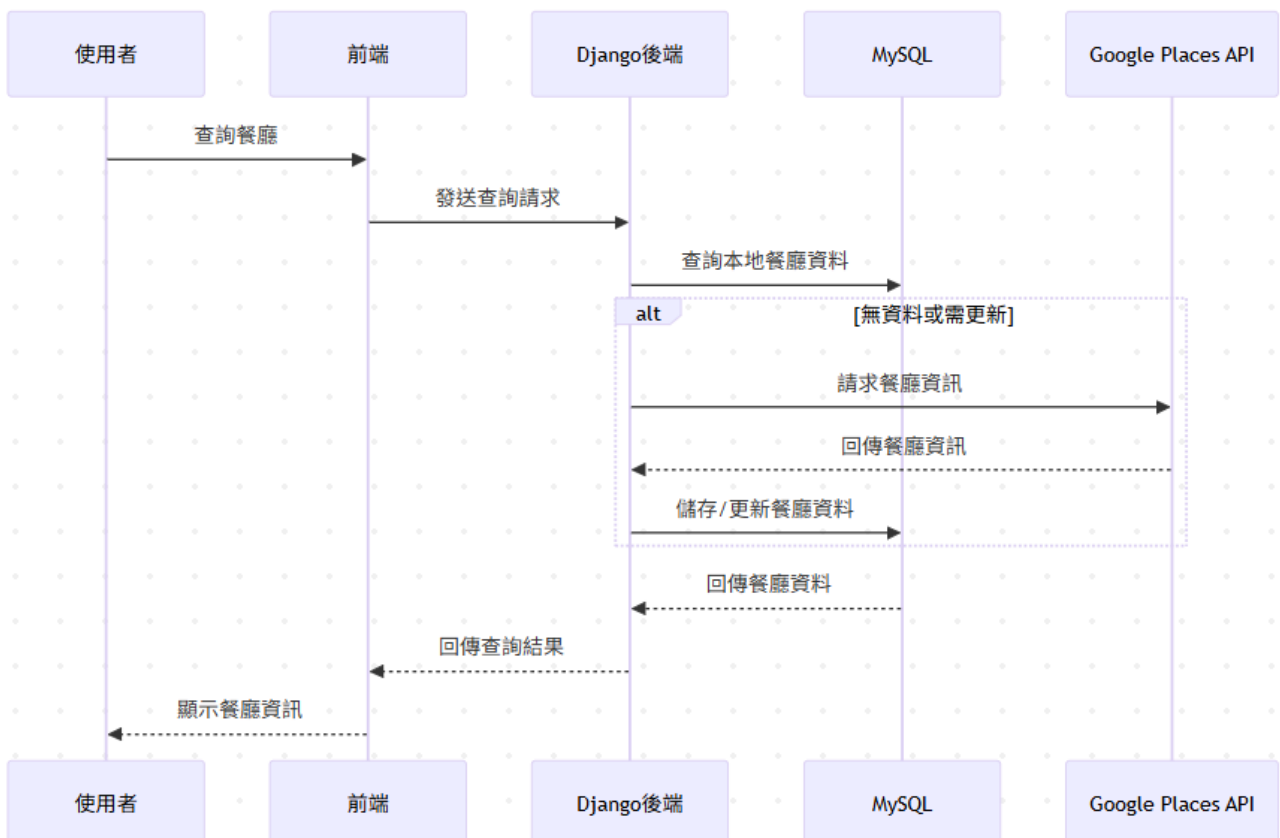
▲圖 5-4-1：分析類別圖

第 6 章 設計模型

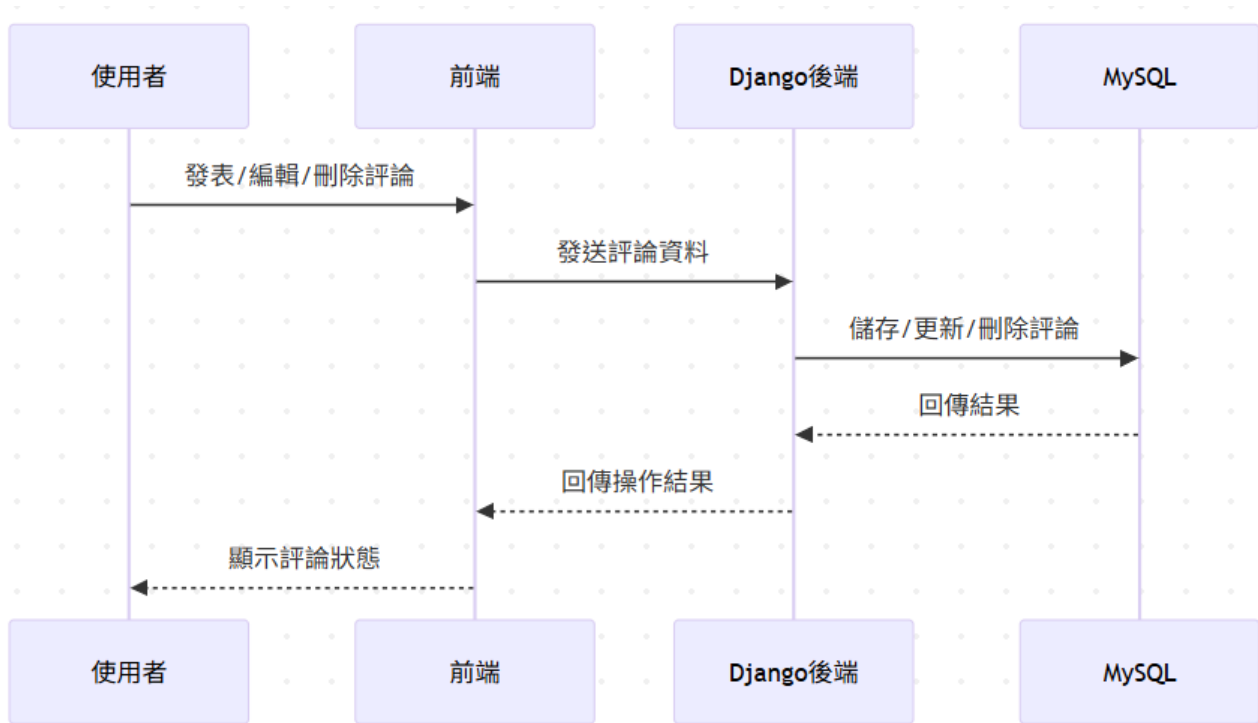
6-1 循序圖(Sequential diagram)



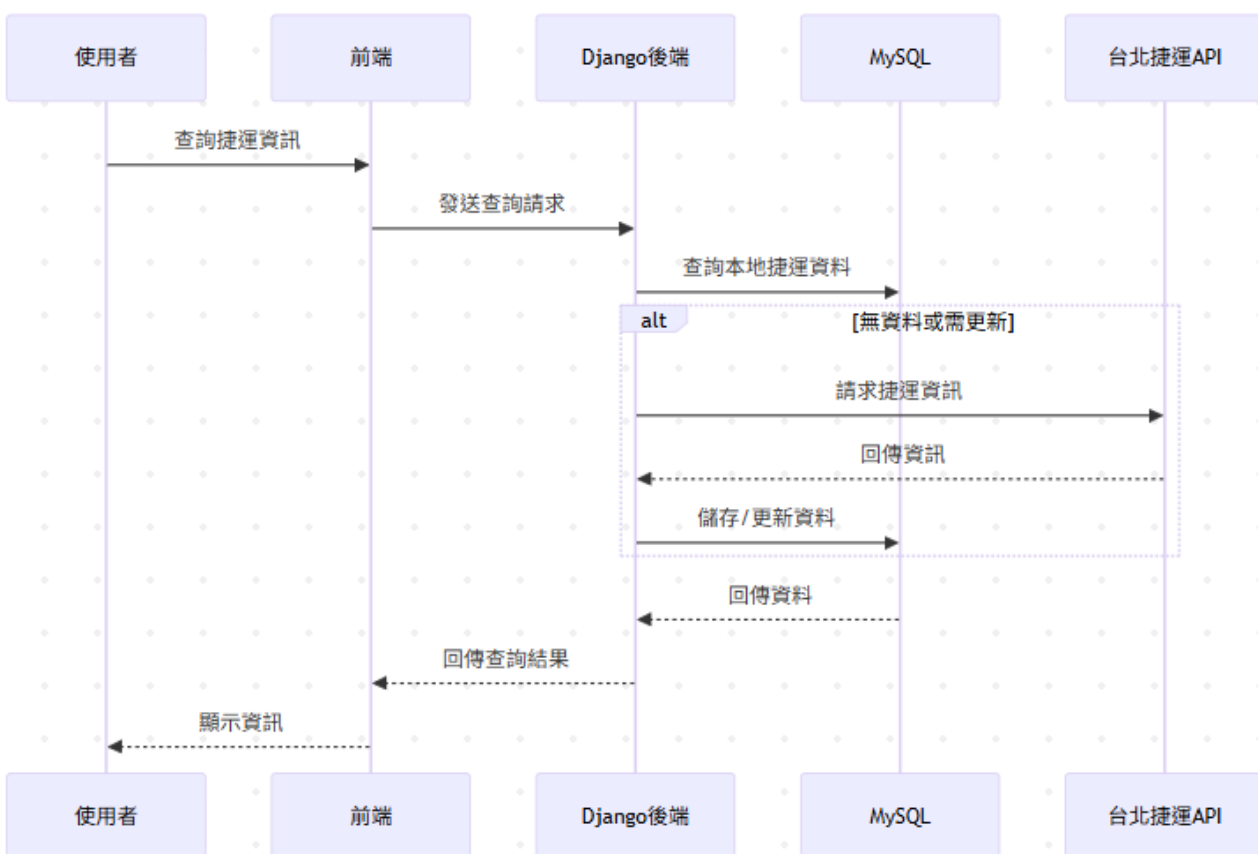
▲圖 6-1-1：循序圖－登入/註冊



▲圖 6-1-2：循序圖－餐廳查詢

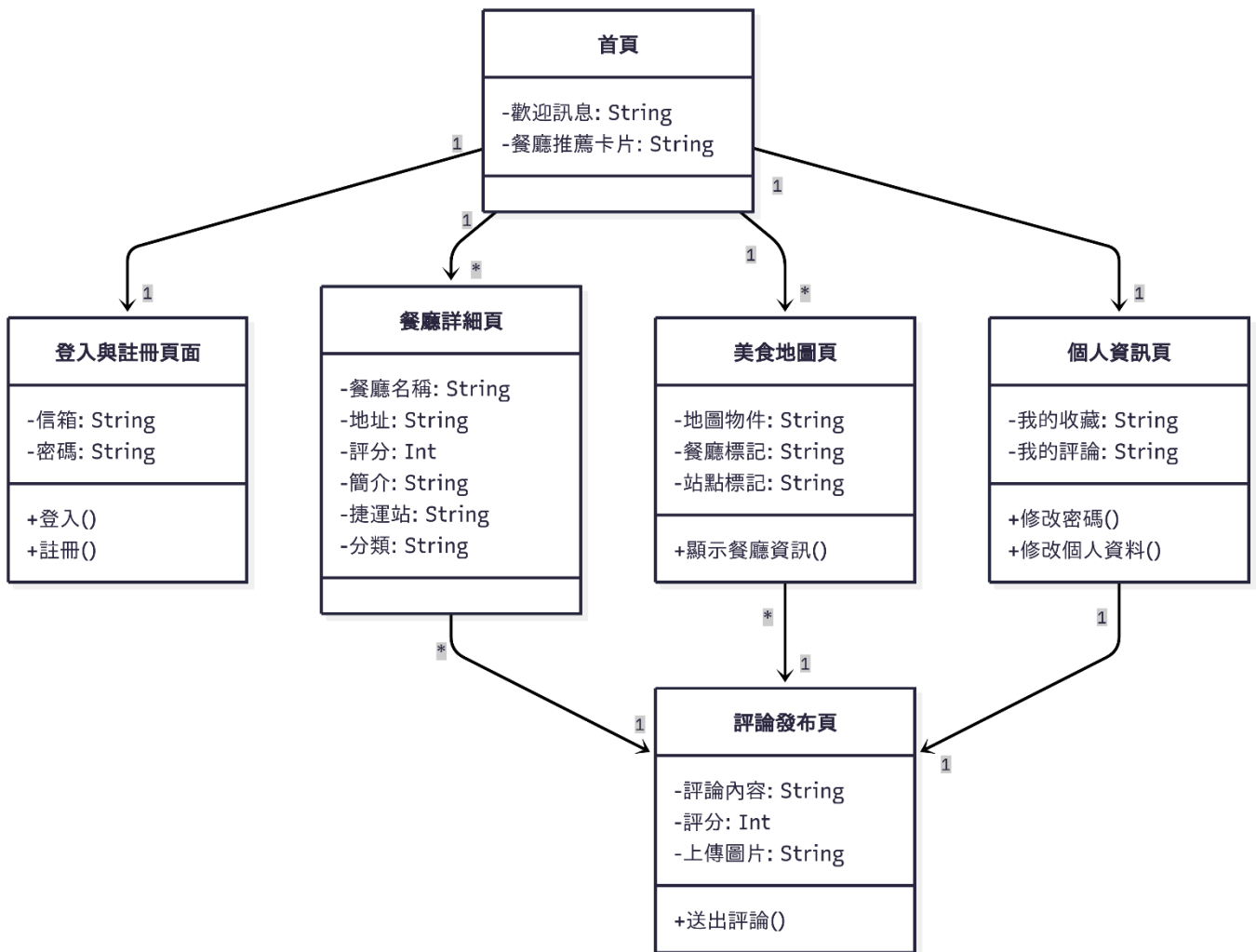


▲圖 6-1-3：循序圖－餐廳評論



▲圖 6-1-4：循序圖－捷運查詢

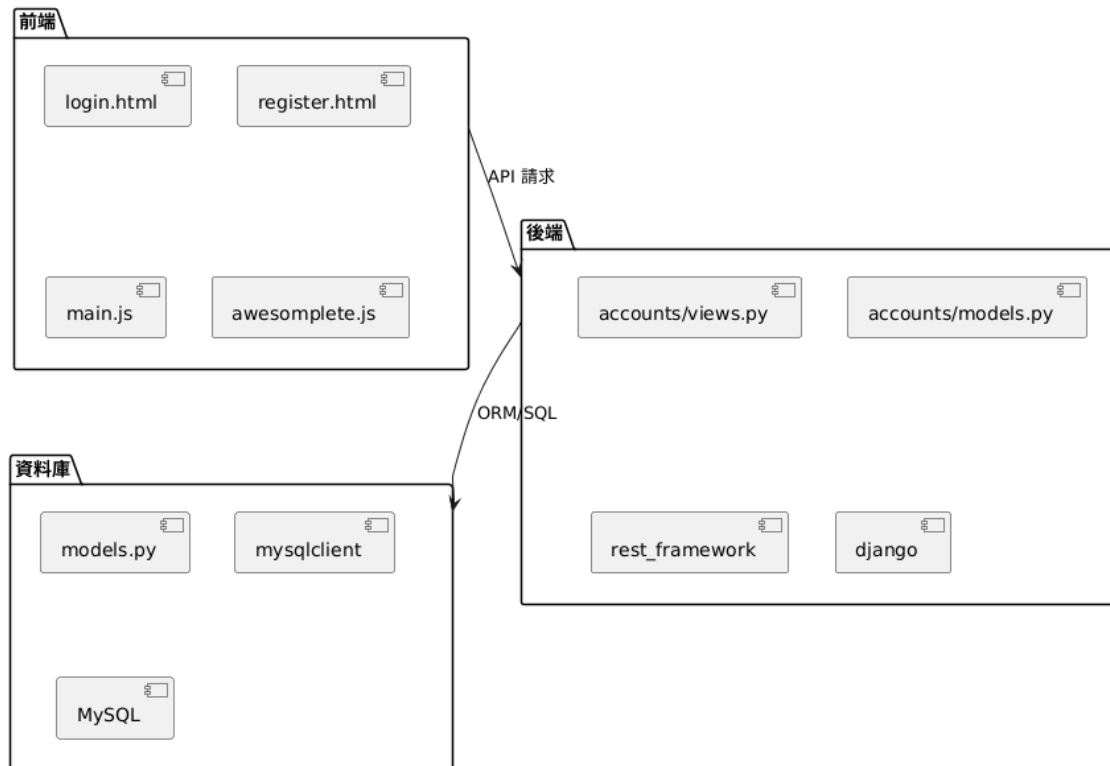
6-2 設計類別圖(Design class diagram)



▲圖 6-2-1：設計類別圖

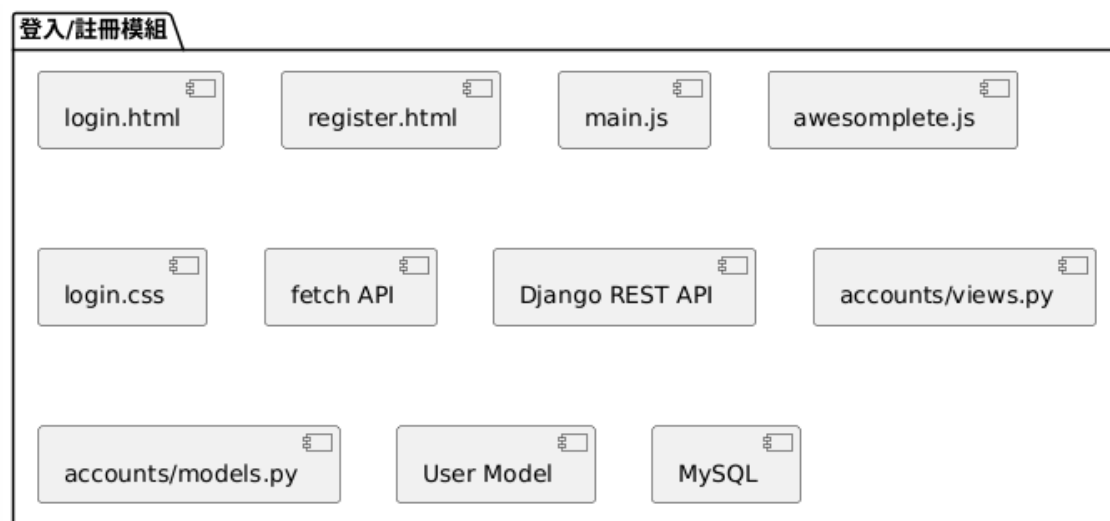
第 7 章 實作模型

7-1 佈署圖(Deployment diagram)

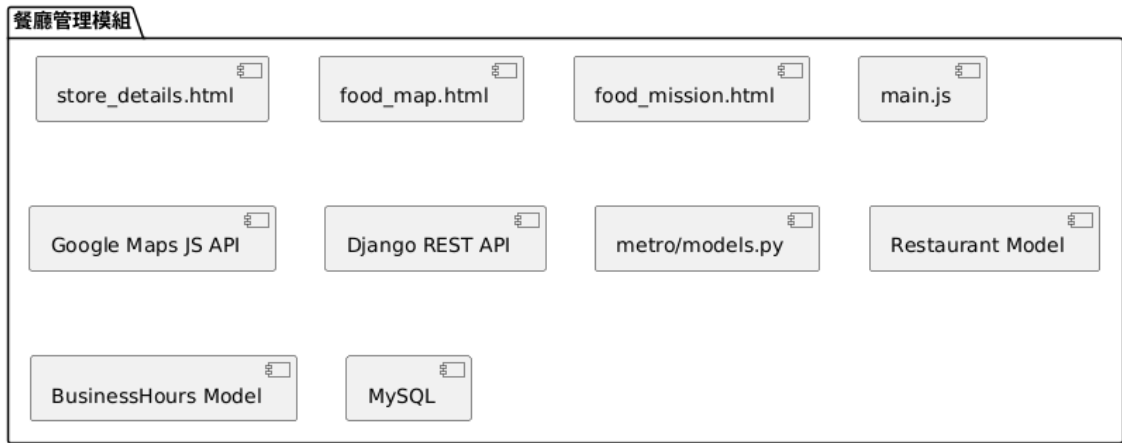


▲圖 7-1-1：佈署圖

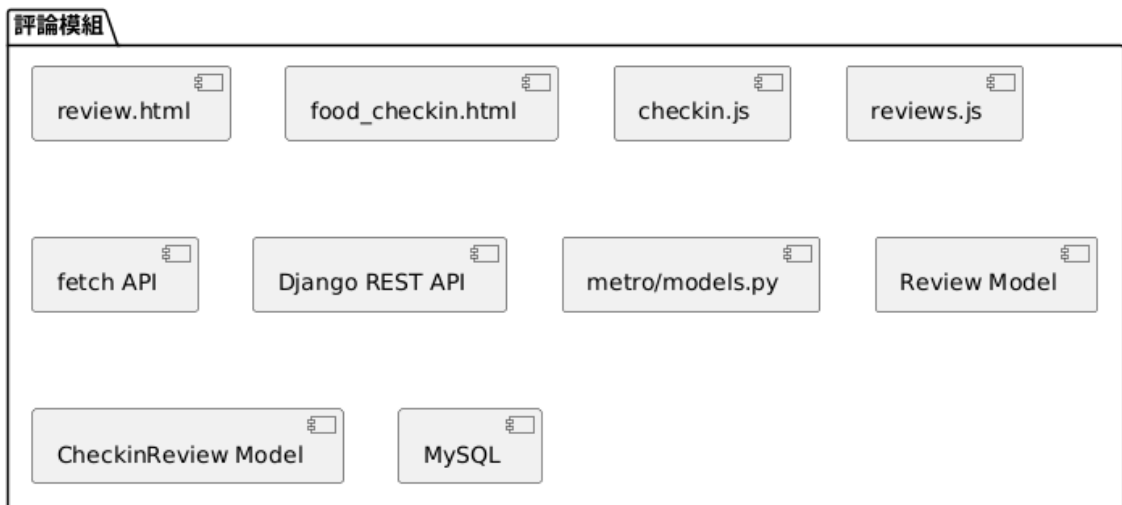
7-2 套件圖(Package diagram)



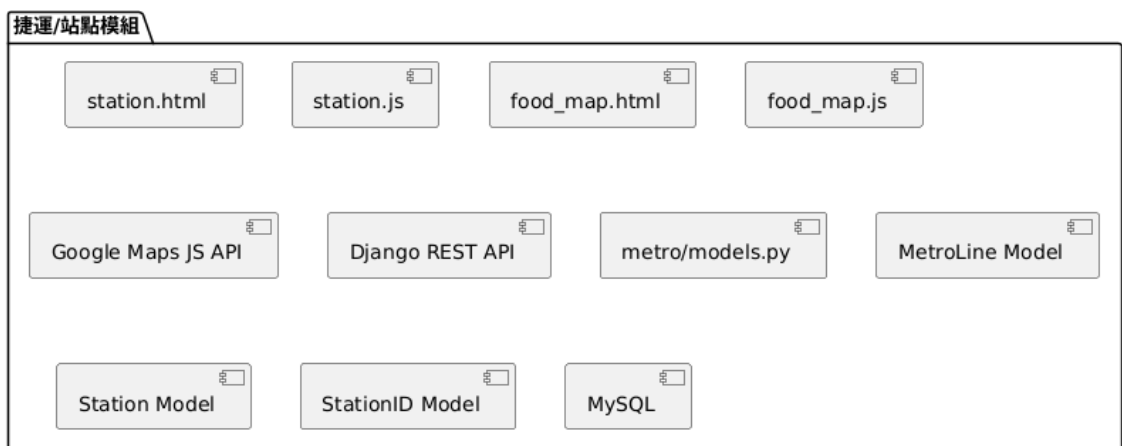
▲圖 7-2-1：套件圖－登入/註冊模組



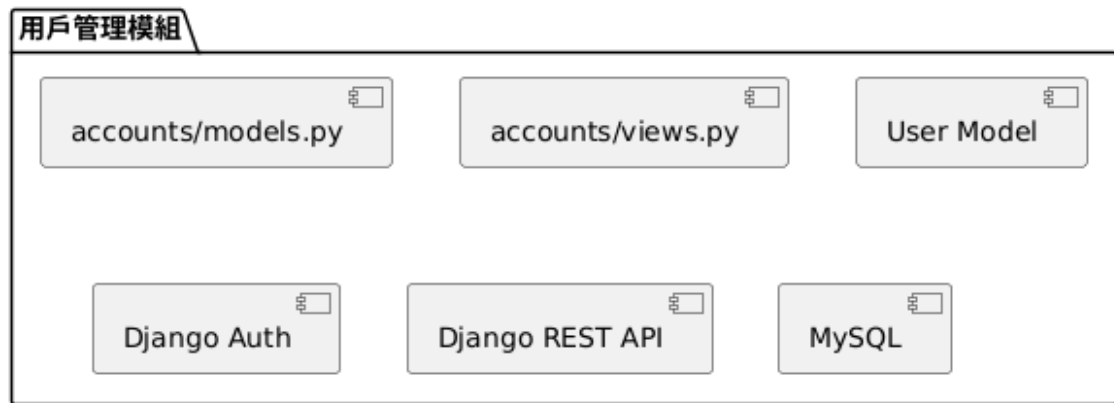
▲圖 7-2-2：套件圖－餐廳管理模組



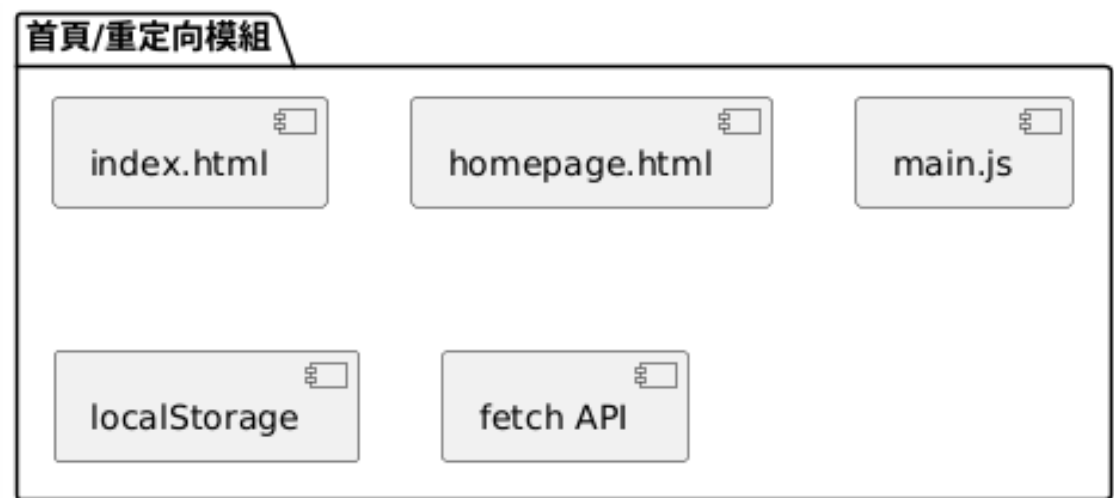
▲圖 7-2-3：套件圖－評論模組



▲圖 7-2-4：套件圖－捷運/站點模組

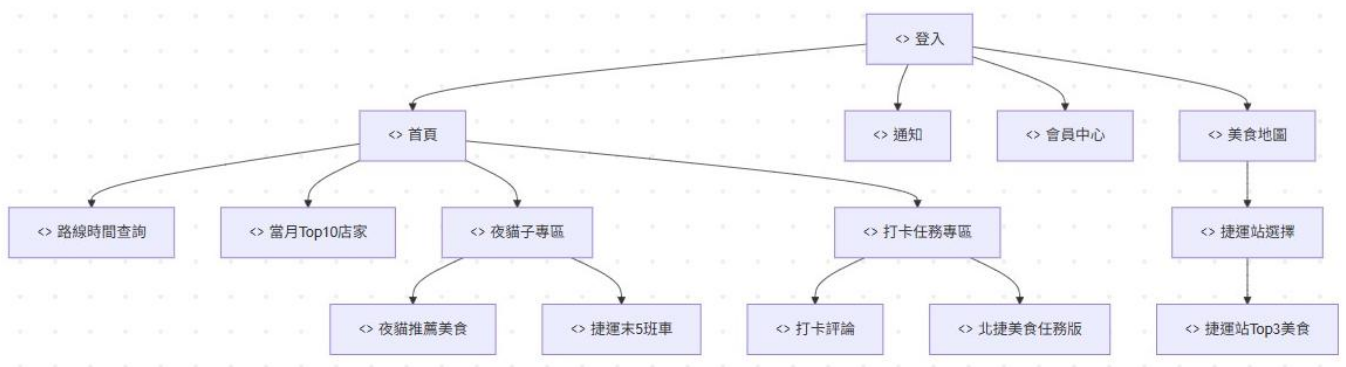


▲圖 7-2-5：套件圖－用戶管理模組



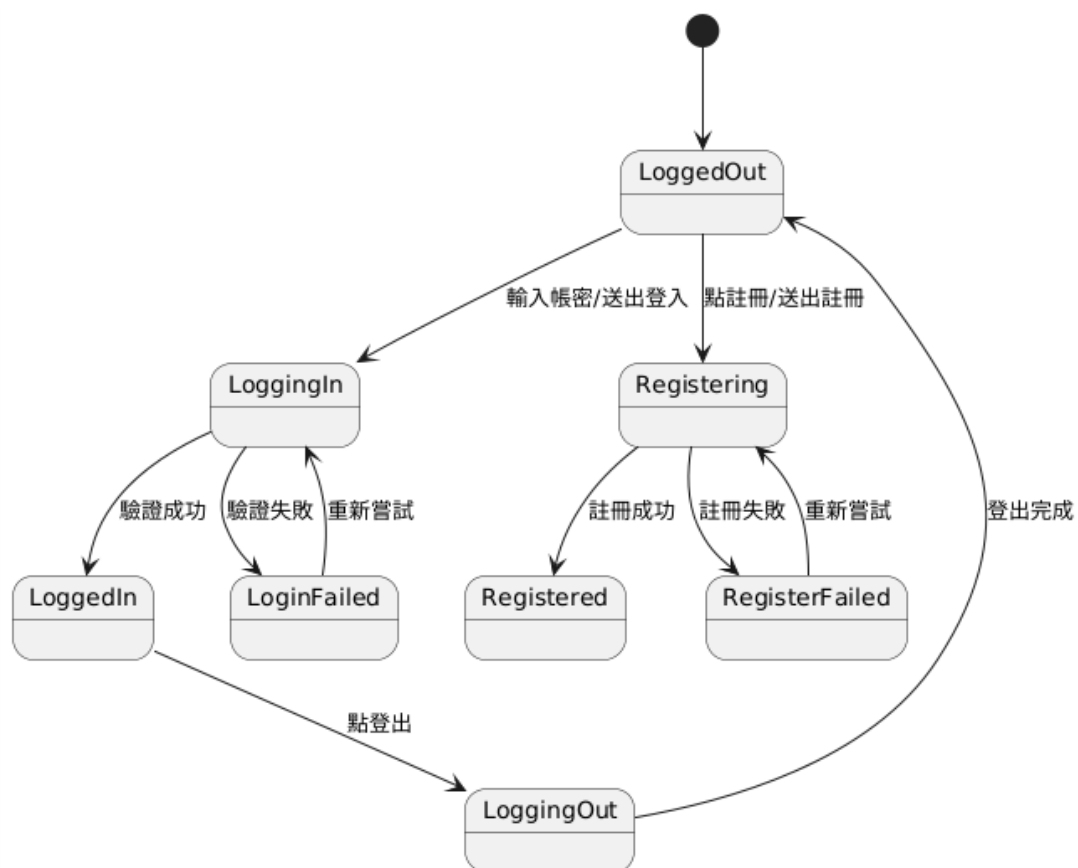
▲圖 7-2-6：套件圖－首頁/重定向模組

7-3 元件圖(Component diagram)

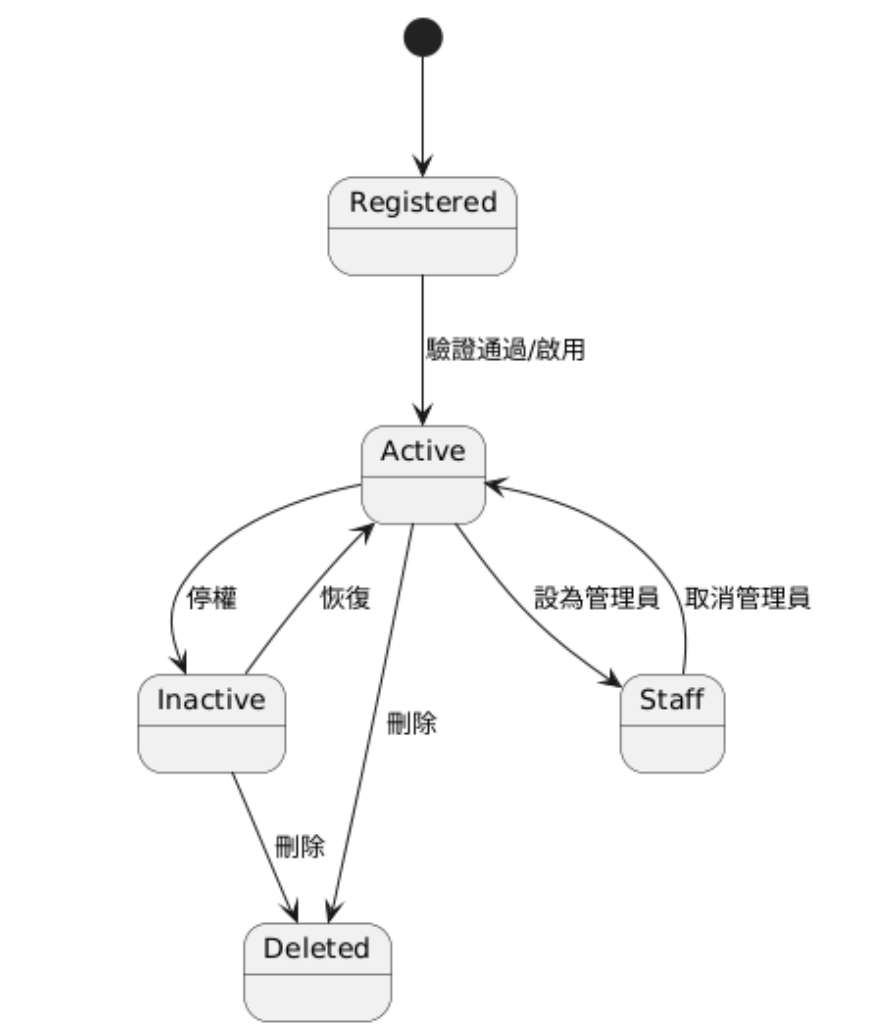


▲圖 7-3-1：元件圖

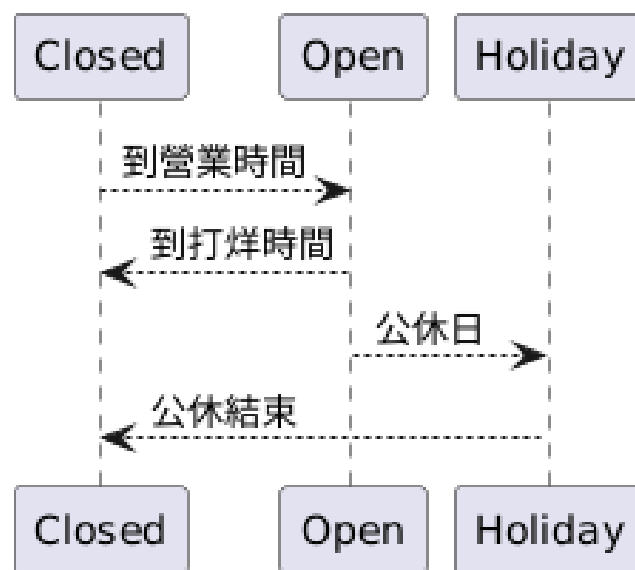
7-4 狀態機(State machine)



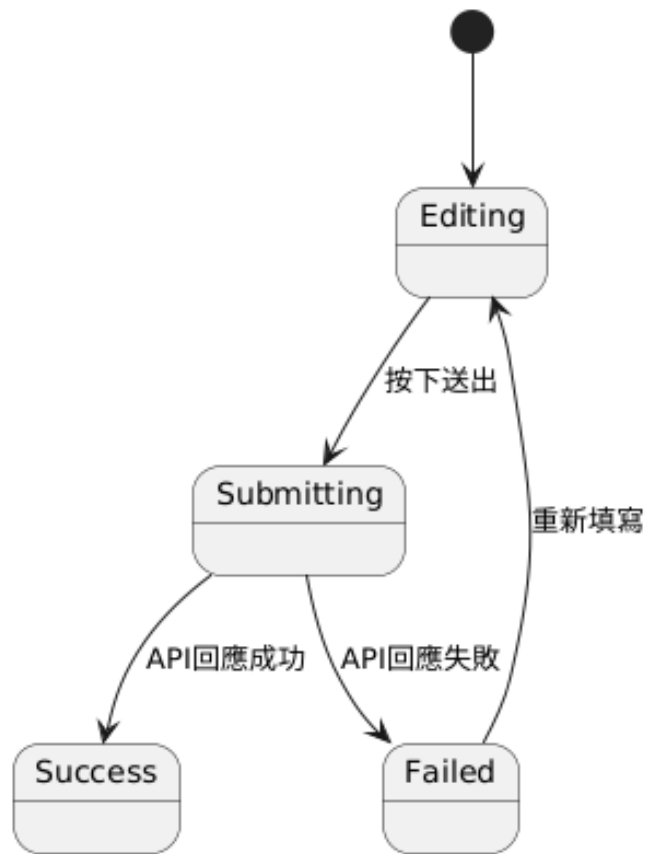
▲圖 7-4-1：狀態機－登入/註冊流程



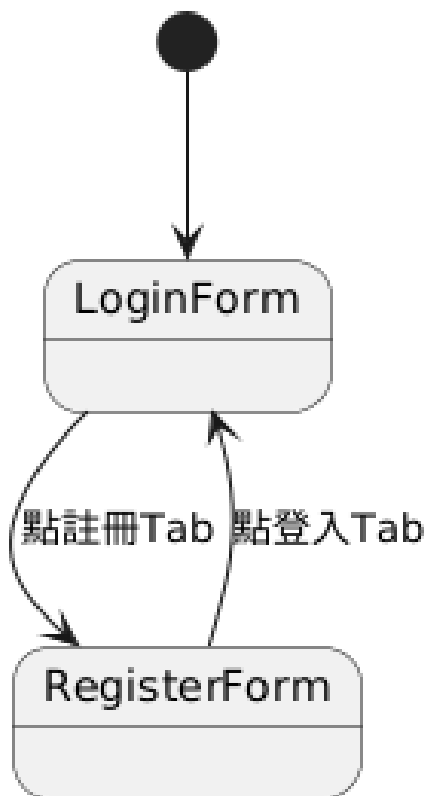
▲圖 7-4-2：狀態機－用戶（帳號啟用/停權/管理員）



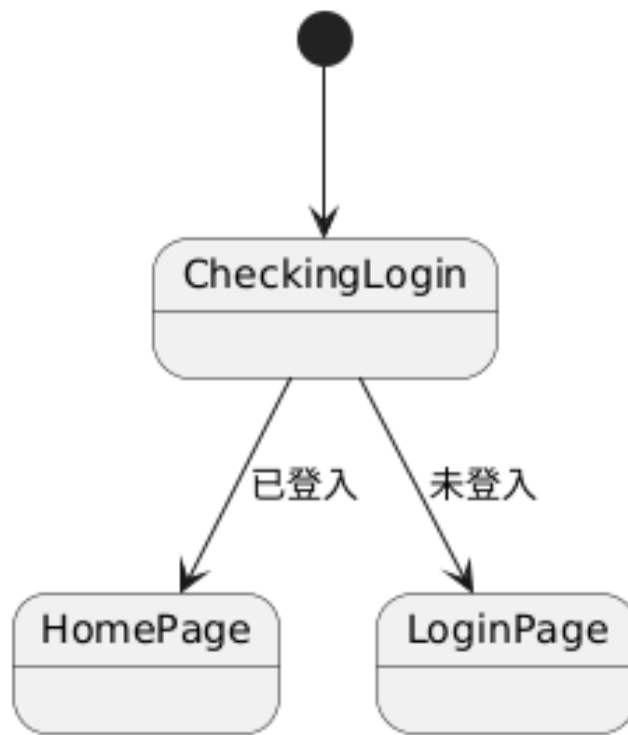
▲圖 7-4-3：狀態機－餐廳營業



▲圖 7-4-4：狀態機－評論發佈流程



▲圖 7-4-5：狀態機－前端頁面/表單切換



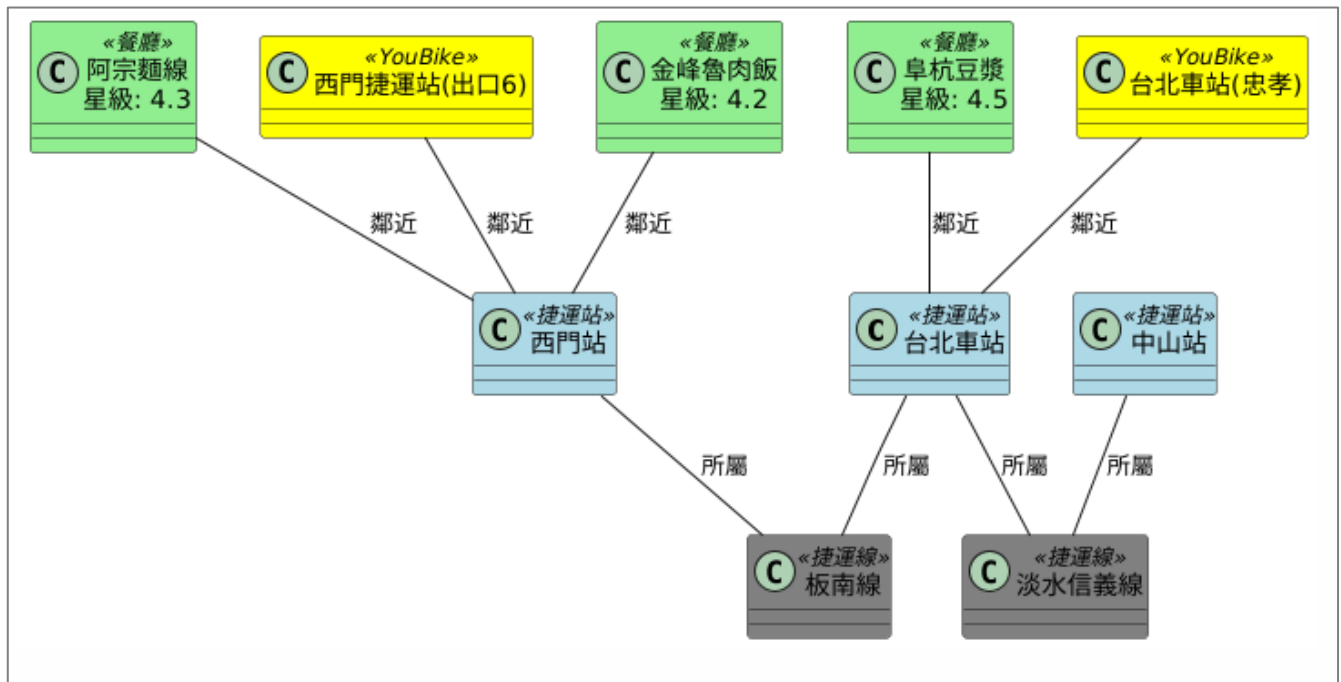
▲圖 7-4-6：狀態機—首頁重定向

第 8 章 資料庫設計

8-1 資料庫關聯表



▲圖 8-1-1：MySQL 資料庫 ER 圖



▲圖 8-1-2：資料庫關聯圖

8-2 表格及其 Meta data

▼表 8-2-1：資料表

編號	資料表英文名稱	中文名稱/用途
01	user	使用者資料
02	metroline	捷運線路
03	station	捷運站
04	restaurant	餐廳
05	businesshours	餐廳營業時間
06	review	餐廳評論
07	checkinreview	打卡評論
08	traininfo	列車到站資訊
09	stationid	車站 ID 對照表

01 user（使用者資料）

▼表 8-2-2：user

中文名稱	使用者資料	資料表編號	01
英文名稱	user	主索引	id
資料檔陳述	紀錄每一位使用者的個人資料及帳號、密碼		
欄位名稱	型態	長度/限制	說明
id	AutoField		主鍵，自動遞增
password	CharField	128	密碼（加密儲存）
last_login	DateTime	可為空	上次登入時間
is_superuser	BooleanField		是否為超級管理員
username	CharField	150,唯一	使用者名稱
first_name	CharField	150	名字
last_name	CharField	150	姓氏
email	EmailField	唯一	電子郵件
is_staff	BooleanField		是否為後台管理員
is_active	BooleanField		帳號是否啟用
date_joined	DateTime		註冊時間
created_at	DateTime		創建時間（你自訂的欄位）
updated_at	DateTime		更新時間（你自訂的欄位）
groups	ManyToMany		所屬群組（權限用）
user_permissions	ManyToMany		個別權限

02 metroline（捷運路線）

▼表 8-2-3：metroline

中文名稱	捷運路線	資料表編號	02
英文名稱	metroline	主索引	id
資料檔陳述	儲存台北捷運的各路線		
欄位名稱	型態	長度/限制	說明
id	AutoField		主鍵

name	CharField	100,唯一	捷運線名稱
color	CharField	20	捷運線顏色
created_at	DateTime		創建時間
updated_at	DateTime		更新時間

03 station（捷運站）

▼表 8-2-4：station

中文名稱	捷運站	資料表編號	03
英文名稱	station	主索引	id
資料檔陳述	儲存台北捷運的各站站名		
欄位名稱	型態	長度/限制	說明
id	AutoField		主鍵
name	CharField	100	站名
metro_line_id	ForeignKey	MetroLine	所屬捷運線
latitude	DecimalField	15,12	緯度
longitude	DecimalField	15,12	經度
station_code	CharField	10	站點代碼
created_at	DateTime		創建時間
updated_at	DateTime		更新時間

04 restaurant（餐廳）

▼表 8-2-5：restaurant

中文名稱	餐廳	資料表編號	04
英文名稱	restaurant	主索引	id
資料檔陳述	儲存餐廳名稱及詳細資料		
欄位名稱	型態	長度/限制	說明
id	AutoField		主鍵
name	CharField	200	店家名稱
address	CharField	500	地址
latitude	DecimalField	15,12	緯度

longitude	DecimalField	15,12	經度
phone	CharField	20,可為空	電話
google_place_id	CharField	100,唯一	GooglePlacesID
rating	DecimalField	3,1,可為空	Google 評分
price_level	IntegerField	可為空	價格等級
website	URLField	可為空	網站
image	CharField	500,可為空	照片參考
created_at	DateTime		創建時間
updated_at	DateTime		更新時間
last_review_update	DateTime	可為空	最後評論更新時間
...	ManyToMany	Station,FoodCategory	關聯捷運站/美食分類

05 businesshours（餐廳營業時間）

▼表 8-2-6：businesshours

中文名稱	餐廳營業時間	資料表編號	05
英文名稱	businesshours	主索引	id
資料檔陳述	儲存餐廳營業資訊		
欄位名稱	型態	長度/限制	說明
id	AutoField		主鍵
restaurant_id	ForeignKey	Restaurant	餐廳
day_of_week	IntegerField		星期幾
open_time	TimeField		開店時間
close_time	TimeField		關店時間
is_closed	BooleanField		是否休息
created_at	DateTime		創建時間
updated_at	DateTime		更新時間

06 review（餐廳評論）

▼表 8-2-7：review

中文名稱	餐廳評論	資料表編號	06
英文名稱	review	主索引	id
資料檔陳述	儲存 Google 的評論		
欄位名稱	型態	長度/限制	說明
id	AutoField		主鍵
restaurant_id	ForeignKey	Restaurant	餐廳
user_id	ForeignKey	User,可為空	使用者
reviewer_name	CharField	100,可為空	評論者名稱
rating	IntegerField		評分
comment	TextField	可為空	評論內容
created_at	DateTime		創建時間
updated_at	DateTime		更新時間

07 checkinreview（打卡評論）

▼表 8-2-8：checkinreview

中文名稱	打卡評論	資料表編號	07
英文名稱	checkinreview	主索引	id
資料檔陳述	儲存打卡任務的評論內容		
欄位名稱	型態	長度/限制	說明
id	AutoField		主鍵
restaurant_name	CharField	100	餐廳名稱
metro_line	CharField	50	捷運線
reviewer_name	CharField	50,預設匿名	評論者名稱
rating	IntegerField		評分
comment	TextField	可為空	評論內容
created_at	DateTime		創建時間
updated_at	DateTime		更新時間

08 traininfo (列車到站資訊)

▼表 8-2-9：traininfo

中文名稱	列車到站資訊	資料表編號	08
英文名稱	traininfo	主索引	id
資料檔陳述	儲存捷運詳細到站資訊		
欄位名稱	型態	長度/限制	說明
id	AutoField		主鍵
train_number	CharField	50,可為空	列車號碼
station_name	CharField	100	站名
destination_name	CharField	100	目的地
count_down	CharField	50	倒數時間
now_date_time	DateTime		當前時間
created_at	DateTime		創建時間

09 stationid (車站 ID 對照表)

▼表 8-2-10：stationid

中文名稱	車站 ID 對照表	資料表編號	09
英文名稱	stationid	主索引	id
資料檔陳述	儲存捷運路線名稱及站點資訊		
欄位名稱	型態	長度/限制	說明
id	AutoField		主鍵
station_name	CharField	100	車站名稱
line_name	CharField	100	線路名稱
station_id	CharField	20,唯一	車站 ID

第 9 章 程式

9-1 元件清單及其規格描述

▼表 9-1-1：元件清單與規格描述表(前端)－網頁介面撰寫

HTML－網頁介面撰寫		
編號	檔案名稱	功能
1-1-1	login.html	註冊及登入
1-1-2	homepage.html	系統首頁
1-1-3	night.html	夜貓子專區
1-1-4	drinks.html	喝酒專區
1-1-5	bars.html	酒吧&夜店
1-1-6	food_checkin.html	打卡任務
1-1-7	food_mission.html	美食任務版
1-1-8	store_details.html	美食任務版－店家詳細資訊
1-1-9	food_map.html	美食地圖
1-1-10	notify.html	通知
1-1-11	Member_center.html	會員中心
1-1-12	edit_profile.html	修改會員資料
1-1-13	favorites_food.html	收藏美食
1-1-14	review.html	Google 評論及地圖

▼表 9-1-2：元件清單與規格描述表(前端)－網頁介面設計

CSS－網頁介面設計		
編號	檔案名稱	功能
1-2-1	login.css	註冊及登入
1-2-2	homepage.css	系統首頁
1-2-3	night.css	夜貓子專區
1-2-4	bars.css	酒吧&夜店
1-2-5	food_checkin.css	打卡任務
1-2-6	food_mission.css	美食任務版

1-2-7	store_details.css	美食任務版－店家詳細資訊
1-2-8	food_map.css	美食地圖
1-2-9	notify.css	通知
1-2-10	Member_center.css	會員中心
1-2-11	edit_profile.css	修改會員資料
1-2-12	favorites_food.css	收藏美食
1-2-13	review.css	Google 評論及地圖

▼表 9-1-3：元件清單與規格描述表(前端)－網頁動畫設計

JavaScript－網頁動畫設計		
編號	檔案名稱	功能
1-3-1	food_map.js	依選捷運站載入餐廳，補 Google 照片，計算站→店距離與步行時間，支援收藏與通知，提供導航/評論連結
1-3-2	station_name.js	動態建立「依路線分組且帶線色樣式」的捷運站下拉選單，可取得所選站的站碼與站名，供後續載入美食、算距離與導航使用
1-3-3	checkin.js	提供餐廳即時搜尋（Awesomplete+後端 API）、可點選 1～5 星評分、依選取餐廳自動帶出捷運站名、以 localStorage 的 username（無則顯示「訪客」）組成打卡資料並 POST 至/api/api/checkin-reviews/，含任務頁與首頁導向
1-3-4	detail.js	用 Google Map Places 定位並帶出評分、評論數、營業時間與照片；從後端載入打卡評論與關鍵字、備援地址；登入者的自家評論顯示會員名稱與頭像，並提供開啟 Google 評論、訂位、分享快捷
1-3-5	mission.js	載入熱門餐廳 → 用 Google Places 補圖資與營業資訊 → 計算距離最近捷運站

		與步行時間並提供導航 → 支援收藏與通知、顯示評分/評論與詳情連結
1-3-6	stationdata.js	站名自動完成+交換、功能鍵導頁；Top10 餐廳補照片與最近站距離/導航；兩欄彩色路線卡顯示查詢結果；首頁顯示台北車站即時列車
1-3-7	drink.js	動態產生捷運站下拉選單，點「查詢」即呼叫後端 API 取得該站末五班車時刻並渲染於頁面
1-3-8	reviews.js	從 URL 暫存取店家，用 Google Places 拉詳情與前 5 評論並標注地圖；若無 id 則 Text Search 以「相似度+距離」挑最佳再呈現
1-3-9	night.js	整合夜貓美食+末五班車：依站名篩餐廳、地圖定位+照片快取、彈窗顯示評分/營業時間/距離並一鍵導航；同頁以站名查詢並呈現該站「末五班車」時刻

▼表 9-1-4：元件清單與規格描述表(後端)－爬蟲

Python－爬蟲		
編號	檔案名稱	功能
2-1-1	import_restaurants.py	爬取餐廳資訊
2-1-2	google_maps_reviews_crawler.py	爬取餐廳評論

▼表 9-1-5：元件清單與規格描述表(後端)－讀取 API

Python－讀取 API		
編號	檔案名稱	功能
2-2-1	urls.py	定義捷運、餐廳、評論、酒吧、通知等 API 路由對應至 views
2-2-2	views.py	提供主要 API，包含餐廳查詢、評論 CRUD、打卡評論、捷運、路線、酒吧、

		通知等
2-2-3	models.py	定義資料模型，包含 Restaurant、Review、CheckinReview、MetroFirstLastTrain 等
2-2-4	serializers.py	序列化模型資料（部分 API 使用）

▼表 9-1-6：元件清單與規格描述表(後端)－Service

Python－Service		
編號	檔案名稱	功能
2-3-1	review_service.py	建立餐廳、打卡評論、回算餐廳評分、同步 Google Places 評分與評論
2-3-2	metro_train_service.py	以 TDX OAuth2 取 token，同步台北、新北首末班車資料並寫入資料庫

▼表 9-1-7：元件清單與規格描述表(後端)－一定時更新資訊

Python－一定時更新資訊		
編號	檔案名稱	功能
2-4-1	update_reviews.py	批次更新 Google Places 餐廳評論與評分
2-4-2	tasks.py	背景任務、排程（若啟用 APScheduler/Crontab）

▼表 9-1-8：元件清單與規格描述表(後端)－使用者資訊管理

Python－使用者資訊管理		
編號	檔案名稱	功能
2-5-1	urls.py	帳號相關路由（登入、註冊、個資）
2-5-2	views.py	帳號 API，包含登入、註冊、取得使用者資訊
2-5-3	models.py	自訂使用者欄位（暱稱、生日、頭像等）

▼表 9-1-9：元件清單與規格描述表(後端)－串接相關網頁

Python－使用者資訊管理		
編號	檔案名稱	功能
2-6-1	urls.py	專案全域路由入口，掛載各 app 的 urls

2-6-2	settings.py	專案設定：資料庫、APP、CORS、REST Framework、金鑰等
-------	-------------	--------------------------------------

9-2 其他附屬之各種元件

▼表 9-2-1：虛擬碼—homepage.html、homepage.css

編號	1-1-2、1-2-2	程式名稱	homepage.html、homepage.css
目的	建立首頁頁面，提供捷運站搜尋、美食地圖導向、熱門店家顯示，以及 RWD 響應式版面設計。		
虛擬碼			
1. 頁面基本結構設定			
➡ 採用 HTML 5 標準結構，語系設定為 zh-TW			
➡ head 區：			
➢ 設定 UTF-8 編碼與 viewport 響應式設定			
➢ 引入 favicon（圖片/logo.png）、CSS 樣式表 static/css/homepage.css			
➢ 載入 Awesomplete 外部套件（自動完成搜尋）			
2. 主要區塊說明			
➡ header 區塊			
➢ 顯示網站 Logo 與標題「MEI 食不打烊 Taipei Metro」			
➢ 使用 .title-with-logo 排版，支援窄螢幕自動換行			
➡ 搜尋列			
➢ 兩個輸入框（出發站、目的站）+「交換」按鈕+「查詢」按鈕			
➢ 使用 Awesomplete 套件實現捷運站自動完成功能			
➢ 採用 .search-bar 容器，支援 RWD			
➡ 功能按鈕區			
➢ 包含三個主要功能按鈕：夜貓子專區、打卡任務、捷運時刻表			
➢ 使用 .features 容器，按鈕有 icon 與文字說明			
➡ TOP 10 店家區			
➢ 使用 .top-shops 容器顯示當月熱門店家			
➢ .shop-list 為橫向可滑動區域			
➢ .shop-item 顯示單一店家資訊，包含圖片與文字			
➡ 固定底部導覽列			

- 使用<nav class="footer">固定在頁面底部
- 提供「首頁、美食地圖、通知、會員中心」4 個導覽按鈕
- 自動根據網址高亮目前頁面

3. CSS 設定重點

➡ RWD 響應式版心

- 使用:root { --app-max: ... }控制版心寬度
- 根據螢幕寬度分別調整手機、平板、桌機版心最大寬度

➡ 全域樣式

- 使用 box-sizing: border-box;統一計算方式
- body 採用置中排列，背景為淺灰色

➡ 功能按鈕區

- display: flex; justify-content: space-around;
- 小螢幕時按鈕允許換行

➡ TOP 10 店家

- 使用橫向 scroll-snap 技術，使滑動更順暢
- 移除 scrollbar 以提升外觀

➡ Footer 導覽列

- 使用 position: fixed;固定在畫面底部
- 寬度與版心同步，支援 iPhone safe area
- 導覽按鈕在當前頁面時會加上.active 樣式變白底

4. JavaScript 功能

- ➡ 監聽 DOMContentLoaded 事件，自動為當前頁面對應的導覽按鈕加上 active 類別
- ➡ 使用 Awesomplete 載入 stationdata.js 提供站名資料
- ➡ 載入 Google Maps Places API (key 已設定)

5. 頁面行為流程

- ➡ 使用者進入首頁 → 顯示標題、搜尋區、功能區
- ➡ 使用者輸入出發站、目的站 → Awesomplete 自動完成捷運站名
- ➡ 點擊「查詢」按鈕 → 顯示查詢結果區域
- ➡ 下方固定導覽列可切換其他功能頁面（地圖、通知、會員中心）

▼表 9-2-2：虛擬碼—Member_center.html、Member_center.css

編號	1-1-11、1-2-10	程式名稱	Member_center.html、 Member_center.css
目的	提供會員使用者查看與編輯個人資料、上傳頭像、快速進入收藏、美食地圖與捷運資訊，並支援 RWD 響應式版面。		
虛擬碼			
1. 頁面基本結構設定			
➤ 採用 HTML 5 標準結構，語系設定為 zh-TW ➤ <head>：設定 favicon、引入 CSS 樣式、內嵌部分彈窗樣式 ➤ <body>：以.container 作為主版心，內容依序為： ➢ 頁面標題 ➢ 個人資料卡（含頭像） ➢ 功能清單區 ➢ 固定底部導覽列 ➢ 編輯個人資料的彈窗（modal） ➤ RWD 設定 ➢ 使用 CSS 變數--app-max 控制不同裝置的版心寬度（手機 480px、平板 720px、桌機 960px、超寬螢幕 1200px） ➢ 透過@media 斷點調整間距與字體大小，確保小螢幕排版正常			
2. 主要區塊說明			
➤ 頁面標題區 ➢ 顯示「會員中心」標題 ➢ 使用.member-title 加粗字體與固定左右 padding，使標題與個資區對齊 ➤ 個人資料卡 ➢ 左側為頭像.avatar-wrap：採圓形容器，支援頭像圖片，右下角浮動相機按鈕（.avatar-edit），可觸發檔案選擇上傳頭像 ➢ 右側為文字.profile-info：顯示會員名稱、Email、生日、電話、性別，「編輯資訊」按鈕會開啟彈窗 ➤ 功能清單區 ➢ 採垂直排列：「收藏美食」→ favorites_food.html、「捷運時刻表」→ 外部			

連結、「捷運末 5 班」→ night.html、「登出」按鈕

➤ 使用 .menu-item 連結樣式，底部以分隔線區分項目

➡ 固定底部導覽列

➤ 使用 position: fixed 固定於畫面底部，寬度隨版心縮放

➤ 提供「首頁、美食地圖、通知、會員中心」4 個導覽按鈕

➤ 自動根據網址高亮目前頁面

➡ 編輯個人資料彈窗

➤ 使用 .modal-backdrop 作為半透明背景

➤ .modal 內包含 Email、姓名、性別、生日、電話等輸入欄位

➤ 「取消」與「儲存」按鈕分別綁定關閉與送出更新 API

3. CSS 設定重點

➡ 採用多組:root 變數管理尺寸（例如頭像大小、間距、按鈕位置）

➡ .avatar-wrap 使用 overflow: visible 讓相機按鈕能「浮在頭像外側」

➡ .footer 透過 transform: translateX(-50%)置中，並支援 RWD

➡ .menu-item 移除超連結底線，並保持全區塊可點擊

4. JavaScript 功能

➡ 登入檢查與提示

➤ checkLogin()：向後端/api/accounts/user/發送請求，確認是否登入

➤ 若未登入，顯示頂部提示條「尚未登入」，並附登入連結

➡ 載入個資

➤ loadProfile()：登入後自動從/api/accounts/profile/取回會員資料，填入資料卡與彈窗欄位

➡ 編輯與更新個資

➤ updateProfile()：將彈窗輸入的資料透過 PUT 請求送到 /api/accounts/profile/

➡ 頭像上傳

➤ uploadAvatar(file)：將選擇的圖片以 FormData 送至 /api/accounts/profile/avatar/，並即時更新頭像顯示

➡ Modal 控制

➤ openModal() / closeModal()控制彈窗顯示與關閉

➤ 點擊背景可關閉彈窗。

➡ 登出功能

- 點擊「登出」按鈕，呼叫/api/accounts/logout/，並清除 localStorage 後跳轉 login.html。

5. 頁面行為流程

- ➡ 使用者進入會員中心 → 頁面觸發 loadProfile() → 檢查登入狀態
- ➡ 若已登入 → 載入並顯示個資（頭像、Email、電話等）
- ➡ 使用者可：
 - 點擊相機圖示 → 選擇圖片 → 即時上傳更換頭像
 - 點擊「編輯資訊」→ 開啟彈窗 → 修改資料 → 儲存更新
- ➡ 點擊底部導覽列按鈕可切換其他頁面
- ➡ 點擊「登出」後跳轉至登入頁

6. 特殊說明

- ➡ 後端 API 伺服器網址固定為 http://140.131.115.112:8000
- ➡ 跨網域情況下，若後端開啟 CSRF 驗證，需設定 CORS + CSRF
- ➡ 所有資料請求皆使用 credentials: 'include'以攜帶登入 Cookie

▼表 9-2-3：虛擬碼—night.html、night.css

編號	1-1-3、1-2-3	程式名稱	night.html、night.css
目的	提供使用者在深夜時段，快速查找捷運站周邊的推薦美食，並查詢該站的末五班捷運車次資訊，同時串接 Google Map 顯示餐廳位置。		
虛擬碼			

1. 頁面基本結構設定

- ➡ 採用 HTML 5 標準結構，語系設定為 zh-TW
- ➡ <head>：
 - 設定網頁標題與 favicon
 - 引入外部 CSS：夜貓/night.css
- ➡ <body>：主要內容包含：
 - 返回首頁按鈕
 - 「喝酒專區」膠囊按鈕
 - 頁首（Logo、標題、副標題、搜尋列）
 - 推薦美食橫向滑動卡片區

- Google Map 區塊
- 捷運末班車查詢區塊
- 尾端載入 JS 與 Google Maps API

2. 主要區塊說明

➡ 返回按鈕 & 喝酒專區

- 返回按鈕：固定在.container 左上角，使用絕對定位，返回 homepage.html
- 喝酒專區按鈕：右上角膠囊風格連結 (pill-link)，導向 drinks.html

➡ 頁首區

- Logo 圖示左上，使用絕對定位置於 header 區塊
- 主標題「MEI 食不打烊」及副標題「夜貓子專區」置於右方，對齊右側
- 搜尋列：圓角白底，左側為搜尋圖示，右側為輸入欄，供使用者輸入關鍵字搜尋捷運站附近美食

➡ 推薦美食區

- 使用.recommend-container 水平滑動容器，內含 9 張.food-card 卡片
- 每張卡片包含：店家圖片、店名 + 商圈介紹、五星評價圖示、「MORE」按鈕
- 設計為手機橫向滑動瀏覽，桌機則可同時顯示多張卡片

➡ Google 地圖區塊

- 預留.map 容器，並使用 Google Maps JavaScript API 載入地圖
- 將來可搭配餐廳資料在地圖上顯示 marker 標記

➡ 捷運末班車查詢區

- 提供下拉選單選擇車站
- 「確定」按鈕觸發查詢
- 查詢結果（末五班車）會顯示於.late-info 容器內

3. CSS 設定重點

➡ 採用 CSS 變數--app-max 控制版心最大寬度，並使用多段 media query：

- 手機：480px
- 平板：720px
- 桌機：960px
- 大螢幕：1200px

➡ .container 使用 position: relative 以便內部按鈕與 Logo 絕對定位

➡ RWD 響應式設計：

- .recommend-container 可橫向滑動以節省小螢幕空間
- .pill-link 按鈕在小螢幕時縮小 padding
- ➡ .food-card 採用 scroll-snap 技術讓滑動停留在卡片中央

4. JavaScript 功能

➡ 登入檢查

- DOMContentLoaded 時檢查 localStorage 中的 isLoggedIn
- 若未登入 → alert('請先登入！') → 導向 login.html

➡ Google Maps 載入

- 使用 Google Maps API Key 與 initMap() callback 載入地圖

➡ 夜貓專區資料

- 透過 night.js 可以補上餐廳資料抓取、末班車查詢 API 串接與地圖標記

5. 頁面行為流程

➡ 使用者進入夜貓子專區 → 檢查登入 → 若未登入跳轉至登入頁

➡ 若登入：

- 顯示推薦美食清單
- 載入 Google 地圖

➡ 使用者可：

- 在搜尋欄輸入站名或關鍵字查詢
- 水平滑動瀏覽推薦美食
- 點擊「MORE」開啟餐廳詳細資料
- 選擇車站並查詢末班車 → 結果顯示在.late-info

➡ 可透過右上膠囊按鈕進入喝酒專區，或點擊返回按鈕回首頁

▼表 9-2-4：虛擬碼—missiom.js

編號	1-3-5	程式名稱	missiom.js
目的	在任務頁載入「熱門打卡餐廳」，計算距離最近捷運站並生成卡片（評分/營業/地址），提供導航與收藏（含通知），並用 Google Places 自動補照片與細節。		
虛擬碼			
1. 基本設定			
☛ 通知模組：notifications_v1 以 localStorage 保存收藏/取消收藏訊息（id、時間、內容、目標店家） ☛ 收藏模組：fav_restaurants_v1 儲存餐廳（以 name@address 鍵去重），toggleFav()切換並更新按鈕樣式 ☛ 站點快取/距離：啟動時抓取所有路線與站點座標為 MRT_STATIONS；Haversine 算直線距離，WALK_M_PER_MIN=80 換算步行分鐘 ☛ 均消換算：Google price_level → 顯示區間（\$100 - 200…\$600 - 1000+）			
2. 資料取得與清洗			
☛ 呼叫/api/api/top-checkin-restaurants/取熱門餐廳 ☛ 過濾測試資料，normalize() 統一欄位：name/rating/review_count/address/website/phone/opening_text/price_text/lat/lng			
3. 卡片渲染（Cards）			
☛ 每筆輸出：圖片（預設占位）、店名、評分+評論數、營業時間、地址、距離列、導航與收藏按鈕、標籤（詳情/均消/官網） ☛ 距離列文案：🚶 距離{最近捷運站}站約 X 公尺（步行約 Y 分鐘） ☛ 導航：有座標 → 以 lat,lng 開 Google Maps；無則以地址查詢 ☛ 收藏：即時切換文字「收藏/已收藏」，並寫入通知列			
4. Google Places 增強（漸進補強）			
☛ textSearch → getDetails：取得照片、完整地址、今日營業字串、price_level、更精準座標。 ☛ 只更新需要的欄位： ➢ 圖片：替換卡片背景圖 ➢ 地址/營業時間/均消：覆寫空白或占位值			

- 座標：更新後重新計算距離
- 收藏狀態依最新 payload 重新判定

5. 例外處理與回退

- ➡ 後端或 Places 失敗：保持占位圖與「—」文案，距離列隱藏但卡片與導航仍可用
- ➡ 空清單與錯誤都有明確提示訊息

6. 擴充、介面連結

- ➡ 「查看詳情」連至 store_details.html?name=... 可銜接評論或地圖詳頁
- ➡ 保留 FAV_PAGE 常數，之後可導向「我的收藏」
- ➡ 站點資料若於前端補上 lat/lng，可跳過首次 API 並提升距離精度

▼表 9-2-5：虛擬碼—stationdata.js

編號	1-3-6	程式名稱	stationdata.js
目的	在首頁整合「捷運站搜尋與路線查詢、Top10 美食卡片（含距離/導航）、即時列車資訊」，提供快速找店與規劃動線的流暢體驗。		
虛擬碼			
1. 基本設定			
➡ 站點資料：內建 stationData（BR/R/G/O/BL 五線站碼與站名），供自動完成與路線著色使用 ➡ 快取：stationCoordCache（站名→座標）與內部 MRT_STATIONS（所有站座標）避免重複查詢 ➡ 常數：步行換算 WALK_M_PER_MIN=80；Haversine 距離函式；Google Maps 導航 URL 產生器			
2. 版位與互動			
➡ 底部導覽高亮：依目前頁檔名自動替對應.nav-btn 加上 active ➡ 功能按鈕（.feature-btn） ➢ night.html 夜貓美食 ➢ food_checkin.html 打卡 ➢ 外連「到站時間」亦支援 data-link（站內）與 data-external（外連）屬性覆寫 ➡ TOP 10 區			

- 首先載入全線站座標 (/api/lines/*/stations/)，建立 MRT_STATIONS
- 取/api/api/restaurants/top10/ → 依評分排序取前 10
- 卡片渲染：先畫出店名/評分/最近捷運站距離+「導航」按鈕，再以 Google Places 只補圖（避免覆蓋既有距離/按鈕）
- 距離文案：🚶 距離{站名}站約 X 公尺（步行約 Y 分）
- 自動插入卡片 CSS（行高、圖片比例、導航鈕樣式等）

🕒 即時列車（首頁熱門站）

- 預設查「台北車站」：呼叫/api/track-info/，彙整各線包含該站的列車，顯示出發站/目的地/方向/倒數與時間

3. 站名搜尋與輸入體驗

- 🕒 Awesomeplete 自動完成：由 stationData 產生「站名(站碼)」清單；支援名稱與站碼同時比對
- 🕒 輸入上色：依站碼前綴 (BR/R/G/O/BL) 即時變更輸入框背景色 (line color)
- 🕒 交換起訖：#swap-btn 互換兩欄內容；防呆避免起訖相同
- 🕒 解析器：parseInput / findBestStation 支援「站名」或「站名(站碼)」與去尾「站」

4. 路線查詢（雙欄卡片）

- 🕒 API：POST /api/api/metro-route/，body：{ start, end }（已清理成標準站名）
- 🕒 方向推斷：依終點關鍵字粗估「東/西/南/北向」
- 🕒 渲染：beautifulRouteCard(result,{startName,endName,dir})
 - 以 result.Path 拆站序，兩欄排列；上方顯示起/迄站與站碼、方向與預估時間
 - 依相鄰站是否同線，用線色著色節點/線段；result.TransferStations 以綠色環標「轉乘」
 - RWD：<420px 自動改為單欄
 - 額外插入樣式使站名右移，避免貼線

5. Google Maps/Places 使用

- 🕒 Top10 照片：textSearch 取首筆照片 URL，只更新；未載入 Places 時用預設圖
- 🕒 導航：若有經緯度 → 一鍵開啟 Google Maps 步行導航；無座標改以地址查詢

6. 錯誤與保護

- ➡ Places 未載入時僅顯示預設圖並記錄警告
- ➡ API/網路失敗時提供替代文案（Top10、即時列車、路線查詢）
- ➡ 站名輸入不可相同；空值提示

7. 可擴充建議

- ➡ stationData 補上 lat/lng 可免初次 API 取座標、提升距離精度
- ➡ Top10 卡片加入「收藏」鈕（沿用既有收藏模組）
- ➡ 即時列車加入站名切換，與首頁搜尋共用自動完成功能

▼表 9-2-6：虛擬碼—metro_train_service.py

編號	2-3-2	程式名稱	metro_train_service.py
目的	整合台北捷運官方 API，提供列車時刻表服務和資料同步功能。		
虛擬碼			

1. 初始化與設置

- ➡ 載入所需的模組和庫：
 - requests：HTTP 請求模組，用於發送 API 請求
 - datetime：日期時間處理模組，用於時間格式轉換
 - django.conf.settings：Django 設定模組，讀取專案設定
 - django.utils.timezone：時間處理模組，處理時區相關功能
 - logging：日誌記錄模組，記錄操作日誌和錯誤訊息
 - ..models：導入自定義模型，用於資料庫操作

2. API 認證功能

- ➡ get_access_token：
 - 設定 token_url 為 TDX API 認證端點
 - 設定 headers：Content-Type 為 application/x-www-form-urlencoded
 - 設定 data：grant_type 為 client_credentials
 - 從 settings 讀取 TDX_CLIENT_ID 和 TDX_CLIENT_SECRET
 - 發送 POST 請求到認證端點
 - 檢查回應狀態碼是否為 200
 - 解析 JSON 回應，提取 access_token
 - 處理認證失敗情況，記錄錯誤日誌
 - 回傳有效的 access token 或 None

3. 捷運時刻表獲取功能

➡ fetch_metro_first_last_trains :

- 調用 get_access_token 獲取 access token
- 驗證 access token 是否有效
- 設定台北捷運 API URL：TRTC 端點
- 設定新北捷運 API URL：NTMC 端點
- 設定 headers：Authorization 為 Bearer token
- 發送 GET 請求到台北捷運 API
- 發送 GET 請求到新北捷運 API
- 解析兩個 API 的 JSON 回應
- 合併台北和新北的資料
- 初始化更新計數器 update_count = 0
- 遍歷所有資料項目進行處理

4. 資料處理和轉換功能

➡ 時間格式轉換：

- 使用 datetime.strptime 解析 FirstTrainTime 字串
- 使用 datetime.strptime 解析 LastTrainTime 字串
- 轉換為 Django TimeField 格式
- 處理時區轉換：使用 fromisoformat 和 replace('Z', '+00:00')

➡ 資料驗證和清理：

- 檢查必要欄位是否存在
- 驗證資料格式是否正確
- 處理缺失或無效資料
- 記錄處理錯誤的日誌

5. 資料庫更新功能

➡ update_or_create 操作：

- 使用 MetroFirstLastTrain.objects.update_or_create()
- 設定唯一識別欄位：line_no、line_id、station_id、destination_station_id、train_type
- 設定 defaults 參數包含所有更新欄位
- 處理創建和更新兩種情況

- 增加更新計數器
- 處理資料庫操作錯誤

➡ 錯誤處理機制：

- 使用 try-except 包裝每個資料處理操作
- 記錄詳細的錯誤日誌
- 繼續處理下一個資料項目
- 不因單一錯誤中斷整個流程

6. 站點查詢功能

➡ get_station_first_last_trains：

- 接收 station_id 參數
- 使用 MetroFirstLastTrain.objects.filter(station_id=station_id) 查詢
- 回傳查詢結果
- 處理查詢錯誤情況
- 記錄查詢操作日誌
- 回傳 None 如果查詢失敗

7. 日誌記錄功能

➡ 成功日誌：

- 記錄成功更新的資料筆數
- 記錄 API 請求成功訊息
- 記錄資料庫操作成功訊息

➡ 錯誤日誌：

- 記錄 API 認證失敗
- 記錄 API 請求失敗
- 記錄資料處理錯誤
- 記錄資料庫操作錯誤
- 包含詳細的錯誤訊息和堆疊追蹤

▼表 9-2-7：虛擬碼—models.py

編號	2-2-3	程式名稱	models.py
目的	定義系統中所有資料庫模型和資料結構，建立完整的資料庫架構。		
虛擬碼			
1. 初始化與設置			
➤ 載入所需的模組和庫：			
➤ django.db.models：Django ORM 核心模組，提供資料庫操作功能			
➤ django.contrib.auth.models：Django 用戶認證模組，提供用戶管理功能			
➤ django.conf.settings：Django 設定模組，讀取專案設定			
➤ django.utils.timezone：時間處理模組，處理時區相關功能			
➤ django.core.validators：資料驗證模組，提供欄位驗證功能			
2. 用戶模型定義（User）			
➤ 繼承 AbstractUser：			
➤ 擴展 Django 內建用戶功能，保留原有認證機制			
➤ 新增個人化欄位：display_name、gender、birthday、phone_number			
➤ 新增頭像上傳功能：avatar 欄位，支援圖片上傳			
➤ 設定 email 為唯一欄位，確保用戶唯一性			
➤ 新增時間戳記：created_at、updated_at 自動記錄時間			
3. 捷運系統模型定義			
➤ MetroLine 模型：			
➤ 管理捷運線路基本資訊：name（線路名稱）、color（線路顏色）			
➤ 設定 name 為唯一欄位，避免重複線路			
➤ 新增時間戳記：created_at、updated_at			
➤ 定義__str__方法，回傳線路名稱			
➤ Station 模型：			
➤ 管理捷運站點詳細資訊：name、latitude、longitude、station_code			
➤ 與 MetroLine 建立外鍵關聯：metro_line 欄位			
➤ 使用 DecimalField 精確儲存座標資訊			
➤ 設定 station_code 為唯一識別碼			
➤ 新增時間戳記：created_at、updated_at			

4. 餐廳系統模型定義

➡ Restaurant 模型：

- 管理餐廳完整資訊：name、address、phone、rating、price_level
- 整合 Google Places API：google_place_id 欄位
- 與 Station 建立多對多關聯：nearby_stations 欄位
- 與 FoodCategory 建立多對多關聯：categories 欄位
- 新增評分相關欄位：rating、price_level
- 新增時間戳記：created_at、updated_at、last_review_update

➡ FoodCategory 模型：

- 管理美食分類：name、description
- 設定 name 為唯一欄位
- 新增時間戳記：created_at、updated_at
- 定義 Meta 類別：verbose_name、ordering

➡ BusinessHours 模型：

- 管理餐廳營業時間：day_of_week、open_time、close_time、is_closed
- 與 Restaurant 建立外鍵關聯
- 定義 DAY_CHOICES 選擇項目
- 設定 unique_together 約束：restaurant + day_of_week

5. 評論系統模型定義

➡ Review 模型：

- 管理餐廳評論：reviewer_name、rating、comment
- 與 Restaurant 建立外鍵關聯：restaurant 欄位
- 與 User 建立外鍵關聯：user 欄位（可為空）
- 設定評分範圍：1~5 星評分
- 新增時間戳記：created_at、updated_at
- 定義 Meta 類別：ordering 按時間倒序

➡ CheckinReview 模型：

- 管理打卡評論：restaurant_name、metro_line、reviewer_name、rating、comment
- 支援匿名評論功能
- 新增時間戳記：created_at、updated_at

- 設定 db_table 自定義表格名稱

6. 互動功能模型定義

➡ CheckinReviewLike 模型：

- 管理評論按讚功能：review、user、created_at
- 與 CheckinReview 建立外鍵關聯
- 與 User 建立外鍵關聯
- 設定 unique_together 約束：review + user

➡ CheckinReviewFavorite 模型：

- 管理評論收藏功能：review、user、created_at
- 與 CheckinReview 建立外鍵關聯
- 與 User 建立外鍵關聯
- 設定 unique_together 約束：review + user

➡ Notification 模型：

- 管理通知系統：recipient、sender、notification_type、title、content
- 定義 NOTIFICATION_TYPES 選擇項目：like、favorite、comment
- 新增 is_read 布林欄位
- 與 CheckinReview 建立外鍵關聯（可為空）
- 新增時間戳記：created_at、updated_at

▼表 9-2-8：虛擬碼—views.py

編號	2-2-2	程式名稱	views.py
目的	提供完整的 RESTful API 端點，處理前端請求並回傳格式化的資料。		
虛擬碼			

1. 初始化與設置

➡ 載入所需的模組和庫：

- rest_framework：Django REST Framework 核心模組
- rest_framework.viewsets：提供 ViewSet 類別
- rest_framework.decorators：提供 API 裝飾器
- rest_framework.response：提供 Response 類別
- rest_framework.permissions：提供權限控制
- django.shortcuts：Django 快捷函數

- `django.core.cache` : Django 快取系統
- `requests` : HTTP 請求模組
- `json` : JSON 處理模組用戶模型定義 (User)

2. ViewSet 類別定義

➤ `MetroLineViewSet` :

- 繼承 `ModelViewSet` , 提供完整 CRUD 操作
- 設定 `queryset = MetroLine.objects.all()`
- 設定 `serializer_class = MetroLineSerializer`
- 設定 `permission_classes = [AllowAny]`
- 支援 `GET/api/metro-lines/`端點

➤ `StationViewSet` :

- 繼承 `ModelViewSet` , 提供完整 CRUD 操作
- 設定 `queryset = Station.objects.all()`
- 設定 `serializer_class = StationSerializer`
- 設定 `permission_classes = [AllowAny]`
- 支援 `GET/api/stations/`端點

➤ `RestaurantViewSet` :

- 繼承 `ModelViewSet` , 提供完整 CRUD 操作
- 設定 `queryset = Restaurant.objects.all()`
- 設定 `serializer_class = RestaurantSerializer`
- 設定 `permission_classes = [AllowAny]`
- 支援 `GET/api/restaurants/`端點

3. 餐廳查詢 API 函數

➤ `get_restaurants_by_station` :

- 使用 `@api_view(['GET'])` 裝飾器
- 設定 `@permission_classes([AllowAny])`
- 從 `request.GET.get('station_id')` 獲取站點 ID
- 驗證站點 ID 是否存在
- 使用 `Station.objects.get(id=station_id)` 查找站點
- 使用 `Restaurant.objects.filter(nearby_stations=station)` 查詢附近餐廳
- 使用 `RestaurantSerializer` 序列化資料

- 回傳 Response(serializer.data)

- 處理 Station.DoesNotExist 異常

➡ get_restaurants_by_line :

- 使用 @api_view(['GET']) 裝飾器

- 從 request.GET.get('line_id') 獲取線路 ID

- 使用 MetroLine.objects.get(id=line_id) 查找線路

- 使用 Station.objects.filter(metro_line=line) 查找線路所有站點

- 使用 Restaurant.objects.filter(nearby_stations__in=stations).distinct() 查詢沿線餐廳

- 使用 RestaurantSerializer 序列化資料

- 回傳 Response(serializer.data)

➡ search_restaurants :

- 使用 @api_view(['GET']) 裝飾器

- 從 request.GET.get('name') 獲取搜尋關鍵字

- 驗證關鍵字是否存在

- 使用 Restaurant.objects.filter(name__icontains=name) 進行模糊搜尋

- 準備回應資料，包含餐廳基本資訊和捷運站資訊

- 回傳格式化的搜尋結果

4. 評論管理 API 函數

➡ create_restaurant_review :

- 使用 @api_view(['POST']) 裝飾器

- 驗證必要欄位：rating

- 檢查評分範圍：1~5

- 調用 create_review_from_frontend 函數

- 處理創建失敗情況

- 回傳成功訊息和評論 ID

➡ create_checkin_review_api :

- 使用 @api_view(['POST']) 裝飾器

- 驗證必要欄位：restaurant_name、metro_line、rating

- 使用 CheckinReview.objects.create() 創建打卡評論

- 設定預設 reviewer_name 為 '匿名'

- 回傳成功訊息和評論 ID

➡ get_restaurant_reviews :

- 使用 @api_view(['GET']) 裝飾器
- 從 request.query_params.get('restaurant_name') 獲取餐廳名稱
- 從 request.query_params.get('limit') 獲取數量限制
- 使用 CheckinReview.objects.filter() 查詢評論
- 使用 order_by('-created_at') 按時間倒序排列
- 格式化回傳資料
- 回傳評論列表

5. 捷運資訊 API 函數

➡ get_track_info :

- 使用 @api_view(['GET']) 裝飾器
- 設定 SOAP 請求標頭：Content-Type、SOAPAction、Host
- 從環境變數獲取 API 帳號密碼：METRO_API_USERNAME、METRO_API_PASSWORD
- 驗證帳號密碼是否存在
- 建立 SOAP XML 請求格式
- 發送 POST 請求到台北捷運 API
- 檢查回應狀態碼
- 解析 JSON 回應資料
- 按照路線分組資料
- 回傳格式化的列車資訊

➡ get_station_last_five_trains :

- 使用 @api_view(['GET']) 裝飾器
- 從 request.GET.get('station_name') 獲取站點名稱
- 驗證站點名稱是否存在
- 使用 MetroLastFiveTrains.objects.filter() 查詢末五班車
- 使用 order_by() 按路線和方向排序
- 整理資料格式
- 回傳末五班車資訊

6. 互動功能 API 函數

➡ toggle_checkin_review_like :

- 使用 @api_view(['POST']) 裝飾器
- 檢查用戶是否已登入
- 使用 get_object_or_404 獲取評論
- 使用 CheckinReviewLike.objects.get_or_create() 處理按讚
- 如果已按讚則刪除，未按讚則創建
- 創建通知給評論作者
- 計算總讚數
- 回傳按讚狀態和數量

➡ toggle_checkin_review_favorite :

- 使用 @api_view(['POST']) 裝飾器
- 檢查用戶是否已登入
- 使用 get_object_or_404 獲取評論
- 使用 CheckinReviewFavorite.objects.get_or_create() 處理收藏
- 如果已收藏則刪除，未收藏則創建
- 創建通知給評論作者
- 計算總收藏數
- 回傳收藏狀態和數量

➡ get_user_notifications :

- 使用 @api_view(['GET']) 裝飾器
- 檢查用戶是否已登入
- 查詢未讀通知：Notification.objects.filter(recipient=user, is_read=False)
- 查詢所有通知：Notification.objects.filter(recipient=user)
- 按時間倒序排列
- 格式化通知資料
- 回傳通知列表和未讀數量

▼表 9-2-9：虛擬碼－serializers.py

編號	2-2-4	程式名稱	serializers.py
目的	將 Django 模型序列化為 JSON 格式，提供 API 資料轉換功能。		
虛擬碼			
1. 初始化與設置 ➡ 載入所需的模組和庫： ➤ rest_framework.serializers：Django REST Framework 序列化器 ➤ rest_framework.fields：提供各種欄位類型 ➤ 自定義模型：從 models.py 導入所有模型類別 ➤ django.db.models：Django ORM 模組 2. 基礎序列化器定義 ➡ MetroLineSerializer： ➤ 繼承 ModelSerializer ➤ 設定 model = MetroLine ➤ 定義 fields = ['id', 'name', 'color', 'created_at', 'updated_at'] ➤ 自動處理模型欄位序列化 ➤ 支援建立和更新操作 ➡ StationSerializer： ➤ 繼承 ModelSerializer ➤ 設定 model = Station ➤ 定義 fields = ['id', 'name', 'latitude', 'longitude', 'station_code', 'metro_line', 'created_at', 'updated_at'] ➤ 使用 depth = 1 自動序列化外鍵關聯 ➤ 包含捷運線路資訊 3. 餐廳序列化器定義 ➡ RestaurantSerializer： ➤ 繼承 ModelSerializer ➤ 設定 model = Restaurant ➤ 定義 fields = ['id', 'name', 'address', 'latitude', 'longitude', 'phone', 'rating', 'price_level', 'website', 'image', 'nearby_stations', 'categories', 'created_at',			

'updated_at']

- 使用 StringRelatedField 序列化多對多關聯
- 新增計算欄位：average_rating、review_count
- 使用 SerializerMethodField 計算平均評分
- 使用 SerializerMethodField 計算評論數量

➡ FoodCategorySerializer :

- 繼承 ModelSerializer
- 設定 model = FoodCategory
- 定義 fields = ['id', 'name', 'description', 'created_at', 'updated_at']
- 支援建立和更新操作

4. 評論序列化器定義

➡ ReviewSerializer :

- 繼承 ModelSerializer
- 設定 model = Review
- 定義 fields = ['id', 'restaurant', 'reviewer_name', 'rating', 'comment', 'created_at', 'updated_at']
- 使用 StringRelatedField 序列化餐廳資訊
- 新增驗證：rating 範圍 1~5
- 支援建立和更新操作

➡ CheckinReviewSerializer :

- 繼承 ModelSerializer
- 設定 model = CheckinReview
- 定義 fields = ['id', 'restaurant_name', 'metro_line', 'reviewer_name', 'rating', 'comment', 'created_at', 'updated_at']
- 新增驗證：rating 範圍 1~5
- 支援建立和更新操作

5. 互動功能序列化器定義

➡ CheckinReviewLikeSerializer :

- 繼承 ModelSerializer
- 設定 model = CheckinReviewLike
- 定義 fields = ['id', 'review', 'user', 'created_at']

- 使用 StringRelatedField 序列化關聯物件
- 支援建立和刪除操作

➤ CheckinReviewFavoriteSerializer :

- 繼承 ModelSerializer
- 設定 model = CheckinReviewFavorite
- 定義 fields = ['id', 'review', 'user', 'created_at']
- 使用 StringRelatedField 序列化關聯物件
- 支援建立和刪除操作

➤ NotificationSerializer :

- 繼承 ModelSerializer
- 設定 model = Notification
- 定義 fields = ['id', 'recipient', 'sender', 'notification_type', 'title', 'content', 'is_read', 'created_at', 'updated_at']
- 使用 StringRelatedField 序列化用戶資訊
- 支援更新已讀狀態

6. 捷運資訊序列化器定義

➤ MetroFirstLastTrainSerializer :

- 繼承 ModelSerializer
- 設定 model = MetroFirstLastTrain
- 定義 fields = ['id', 'line_no', 'line_id', 'station_id', 'station_name', 'station_name_en', 'trip_head_sign', 'destination_station_id', 'destination_station_name', 'destination_station_name_en', 'train_type', 'first_train_time', 'last_train_time', 'monday', 'tuesday', 'wednesday', 'thursday', 'friday', 'saturday', 'sunday', 'national_holidays', 'src_update_time', 'update_time', 'version_id']
- 支援查詢和篩選操作

➤ MetroLastFiveTrainsSerializer :

- 繼承 ModelSerializer
- 設定 model = MetroLastFiveTrains
- 定義 fields = ['id', 'line_no', 'line_id', 'station_id', 'station_name', 'station_name_en', 'trip_head_sign', 'destination_station_id',

'destination_station_name', 'destination_station_name_en', 'train_type',
'train_time', 'train_sequence', 'monday', 'tuesday', 'wednesday', 'thursday',
'friday', 'saturday', 'sunday', 'national_holidays', 'src_update_time',
'update_time', 'version_id']

- 支援查詢和篩選操作

7. 自定義序列化器方法

➡ get_average_rating :

- 計算餐廳平均評分
- 使用 aggregate 方法計算平均值
- 處理無評分情況
- 回傳格式化評分

➡ get_review_count :

- 計算餐廳評論數量
- 使用 count 方法統計評論
- 回傳評論總數

➡ validate_rating :

- 驗證評分範圍
- 檢查評分是否在 1~5 範圍內
- 回傳驗證錯誤或通過

▼表 9-2-10：虛擬碼—review_service.py

編號	2-3-1	程式名稱	review_service.py
目的	處理評論相關業務邏輯，整合 Google Places API 提供評論同步功能。		
虛擬碼			

1. 初始化與設置

➡ 載入所需的模組和庫：

- requests：HTTP 請求模組，用於發送 API 請求
- datetime：日期時間處理模組，用於時間格式轉換
- timedelta：時間差計算模組，用於時間比較
- django.conf.settings：Django 設定模組，讀取專案設定
- django.utils.timezone：時間處理模組，處理時區相關功能

- `django.db.models`：Django ORM 模組，用於資料庫操作
- `django.db.models.Avg`：聚合函數，用於計算平均值
- `logging`：日誌記錄模組，記錄操作日誌和錯誤訊息
- `..models`：導入自定義模型，用於資料庫操作

2. 評論創建功能

➡ `create_review_from_frontend`：

- 接收 `restaurant_id` 和 `review_data` 參數
- 驗證評分範圍：檢查 `rating` 是否為 1~5 的整數
- 記錄無效評分的錯誤日誌
- 使用 `Restaurant.objects.get(id=restaurant_id)` 查找餐廳
- 處理 `Restaurant.DoesNotExist` 異常
- 使用 `Review.objects.create()` 創建評論記錄
- 設定 `reviewer_name` 預設值為 '匿名'
- 設定 `comment` 預設值為空字串
- 設定 `created_at` 為當前時間

3. 餐廳評分更新功能

➡ 更新餐廳統計資料：

- 使用 `Review.objects.filter(restaurant=restaurant)` 查詢所有評論
- 使用 `reviews.count()` 計算評論總數
- 使用 `reviews.aggregate(avg_rating=models.Avg('rating'))` 計算平均評分
- 更新 `restaurant.total_ratings` 欄位
- 更新 `restaurant.rating` 欄位
- 使用 `restaurant.save()` 儲存變更
- 記錄成功創建評論的日誌

4. 打卡評論功能

➡ `create_checkin_review`：

- 接收 `restaurant_id` 和 `review_data` 參數
- 驗證評分範圍：檢查 `rating` 是否為 1~5 的整數
- 記錄無效評分的錯誤日誌
- 使用 `Restaurant.objects.get(id=restaurant_id)` 查找餐廳
- 處理 `Restaurant.DoesNotExist` 異常

- 使用 `CheckinReview.objects.create()` 創建打卡評論
- 設定 `reviewer_name` 預設值為 '匿名'
- 設定 `comment` 預設值為空字串
- 設定 `created_at` 為當前時間
- 記錄成功創建打卡評論的日誌

5. 評論查詢功能

➡ `get_checkin_reviews` :

- 接收 `restaurant_id` 和 `limit` 參數
- 使用 `CheckinReview.objects.filter(restaurant_id=restaurant_id)` 查詢評論
- 使用 `order_by('-created_at')` 按時間倒序排列
- 使用 `[:limit]` 限制回傳數量
- 格式化回傳資料為字典列表
- 包含 `id`、`reviewer_name`、`rating`、`comment`、`created_at` 欄位
- 使用 `strftime` 格式化時間字串
- 處理查詢錯誤情況
- 回傳空列表如果查詢失敗

6. Google Places API 整合功能

➡ `update_restaurant_reviews` :

- 查詢需要更新的餐廳：`last_review_update` 為空或超過 6 小時
- 檢查是否有餐廳需要更新
- 記錄需要更新的餐廳數量
- 遍歷每個需要更新的餐廳
- 構建 Google Places API 請求 URL
- 設定請求參數：`place_id`、`fields`、`key`、`language`
- 發送 GET 請求到 Google Places API
- 檢查 API 回應狀態

7. Google Places API 資料處理

➡ 解析 API 回應 :

- 檢查 `data['status']` 是否為 'OK'
- 記錄 API 請求失敗的警告日誌
- 更新餐廳評分：`data['result']['rating']`

- 更新評論總數：`len(data['result'].get('reviews', []))`

- 儲存餐廳變更

➡ 處理評論資料：

- 遍歷 `data['result']['reviews']` 中的每個評論

- 檢查評論是否已存在：使用 `reviewer_name` 和 `created_at` 查詢

- 如果評論不存在則創建新記錄

- 使用 `datetime.fromtimestamp` 轉換時間戳記

- 創建 Review 記錄：`restaurant`、`reviewer_name`、`rating`、`comment`、`created_at`

➡ 更新最後更新時間：

- 設定 `restaurant.last_review_update` 為當前時間

- 儲存餐廳變更

- 記錄成功更新評論的日誌

8. 錯誤處理和日誌記錄

➡ 異常處理：

- 使用 `try-except` 包裝每個操作

- 處理 `Restaurant.DoesNotExist` 異常

- 處理 API 請求異常

- 處理資料庫操作異常

- 記錄詳細的錯誤日誌

➡ 日誌記錄：

- 記錄成功操作的訊息

- 記錄錯誤操作的詳細資訊

- 包含相關的參數和錯誤訊息

- 使用適當的日誌等級（`info`、`warning`、`error`）

第 10 章 測試模型

10-1 測試計畫

本系統所需的硬體為瀏覽器 iOS 全系列及 Chrome 140.0.7339 以上的版本。在系統程式開發設計完成後，我們將程式存在虛擬機後端後於手機上網頁進行測試，確認每項功能是否能夠執行成功。

- (1) 會員註冊：確認使用者能夠註冊。
- (2) 會員登入：確認使用者能夠登入到首頁。
- (3) 更改會員資料：修改使用者儲存的最新個人資料。
- (4) 站點查詢：查詢站點之間的路線資訊。
- (5) 當月 TOP 10 店家：推薦當月評論星數前 10 的店家資訊。
- (6) 夜貓子專區：夜間使用者以捷運站名尋找美食及捷運末班車資訊。
- (7) 喝酒專區：酒吧和酒店營業詳細資訊及代駕服務。
- (8) 打卡任務評論：透過打卡任務來蒐集使用者的評論內容。
- (9) 美食任務版：查詢所有店家的評論及營業詳細資訊。
- (10) 捷運時刻表：提供北捷各站點詳細到站時間。
- (11) 美食地圖：以捷運站名查詢各站周邊美食資訊。
- (12) 收藏及通知：收藏美食後會通知收藏的相關訊息。

10-2 測試個案與測試結果

▼表 10-2-1：測試結果－會員註冊

功能編號	(1)	功能名稱	會員註冊
測試目標	確認使用者能夠註冊		
測試作業	1. 輸入信箱與密碼 2. 驗證資料庫是否有重複的會員資料 3. 若無重複的會員資料則直接註冊該筆會員資料		
測試結果	成功		

▼表 10-2-2：測試結果－會員登入

功能編號	(2)	功能名稱	會員登入
測試目標	確認使用者能夠登入到首頁		
測試作業	1. 輸入信箱與密碼 2. 驗證資料庫是否有重複的會員資料 3. 若查無符合的會員資料無法登入，需先註冊帳號		
測試結果	成功		

▼表 10-2-3：測試結果－更改會員資料

功能編號	(3)	功能名稱	更改會員資料
測試目標	修改使用者儲存的最新個人資料		
測試作業	1. 點選【會員中心】頁面中的編輯資訊 2. 是否能夠更改會員資料並儲存 3. 再次查看會員資料是否為更改過的資料		
測試結果	成功		

▼表 10-2-4：測試結果－站點查詢

功能編號	(4)	功能名稱	站點查詢
測試目標	查詢站點之間的路線資訊		
測試作業	1. 點選【首頁】頁面 2. 站點選單點選後按下查詢 3. 查看站點與路線的結果		
測試結果	成功		

▼表 10-2-5：測試結果－當月 TOP 10 店家

功能編號	(5)	功能名稱	當月 TOP 10 店家
測試目標	推薦當月評論星數前 10 的店家資訊		
測試作業	1. 點選【首頁】頁面 2. 滑至最下方 3. 查看店家詳細資訊		
測試結果	成功		

▼表 10-2-6：測試結果－夜貓子專區

功能編號	(6)	功能名稱	夜貓子專區
測試目標	夜間使用者以捷運站名尋找美食及捷運末班車資訊		
測試作業	1. 點選【夜貓子專區】頁面 2. 搜尋欄點選欲查詢站名 3. 查看站點周邊夜間美食詳細資訊 4. 選擇欲查詢的站名末五班車 5. 查看末五班車資訊		
測試結果	成功		

▼表 10-2-7：測試結果－喝酒專區

功能編號	(7)	功能名稱	喝酒專區
測試目標	酒吧和酒店營業詳細資訊及代駕服務		
測試作業	1. 點選【夜貓子專區】頁面 2. 點選右上角【喝酒專區】按鈕 3. 點選【  】進入酒吧&夜店查詢頁面 4. 選擇欲查看站點周邊酒吧&夜店營業詳細資訊 5. 點選【  】進入代駕頁面即跳轉至 Uber 6. 輸入上車地及下車地		
測試結果	成功		

▼表 10-2-8：測試結果－打卡任務評論

功能編號	(8)	功能名稱	打卡任務評論
測試目標	透過打卡任務來蒐集使用者的評論內容		
測試作業	1. 點選【首頁】頁面 2. 點選【打卡任務】按鈕 3. 依序輸入相對應的資料後按下送即可完成		
測試結果	成功		

▼表 10-2-9：測試結果－美食任務版

功能編號	(9)	功能名稱	美食任務版
測試目標	查詢所有店家的評論及營業詳細資訊		
測試作業	1. 點選【打卡任務】按鈕 2. 點選中間【美食任務版】按鈕 3. 即可查看店家的相關資訊，也可看到其他用戶的評論		
測試結果	成功		

▼表 10-2-10：測試結果－捷運時刻表

功能編號	(10)	功能名稱	捷運時刻表
測試目標	提供北捷各站點詳細到站時間		
測試作業	1. 點選【捷運時刻表】按鈕 2. 跳轉至北捷各路線詳細站點到站時間		
測試結果	成功		

▼表 10-2-11：測試結果－美食地圖

功能編號	(11)	功能名稱	美食地圖
測試目標	以捷運站名查詢各站周邊美食資訊		
測試作業	1. 點選【美食地圖】頁面 2. 選擇欲查詢站點名稱，即可出現該站 TOP 3 美食資訊 3. 如要查看更多該站美食，可下滑點選【查看更多美食】		
測試結果	成功		

▼表 10-2-12：測試結果－收藏及通知

功能編號	(12)	功能名稱	收藏及通知
測試目標	收藏美食後會通知收藏的相關訊息		
測試作業	1. 在【美食任務版】、【美食地圖】點選收藏按鈕後會跳通知在【通知】頁面 2. 點選【會員中心】頁面選擇【收藏美食】按鈕 3. 該頁面會出現先前收藏的店家資訊		
測試結果	成功		

第 11 章 操作手冊

11-1 系統元件

▼表 11-1-1：系統之元件資訊

元件名稱	MEI 食不打烊
費用	免費
版本需求	iOS 全系列、Android 8 以上
網路需求	Wifi / 4G 以上網路
支援語言	繁體中文

第 12 章 使用手冊

註冊頁面



首次登入的使用者尚未有帳號，需先註冊，輸入電子郵件、密碼及確認密碼後，按下註冊，即可註冊成功。

▲圖 11-1-1：註冊頁面

登入頁面



註冊完畢後，點擊【登入】，輸入帳號及密碼後按下登入，即可進入到網頁。

▲圖 11-1-2：登入頁面

首頁頁面



▲圖 11-1-3：首頁頁面

登入後就會直接進入到首頁，上方為捷運路線查詢、中間為熱門站點即時資訊、下方三個按鈕為此系統最為重要的功能。

站點查詢



a.選擇出發站及目的地

b.查詢結果

使用者可使用站點查詢來搜尋欲出發至目的地的路線，可得知需花費多少乘車時間（不包括換乘、等車時間），以及如需換乘該在哪站換乘。

▲圖 11-1-4：站點查詢

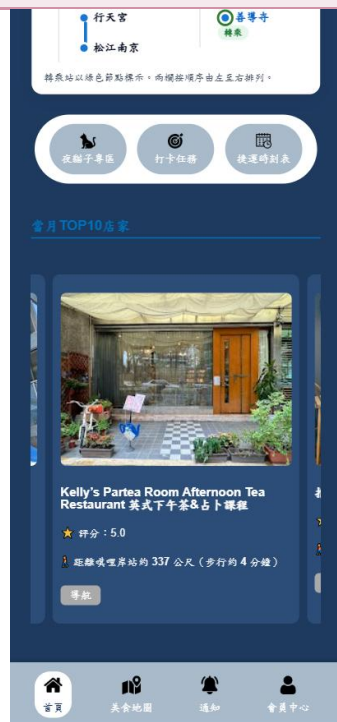
熱門站即時資訊



▲圖 11-1-5：熱門站即時資訊

使用者如果不知道要去哪裡時，我們有提供以台北車站為出發地，至各熱門目的地站點，例如：淡水站、北投站等等……

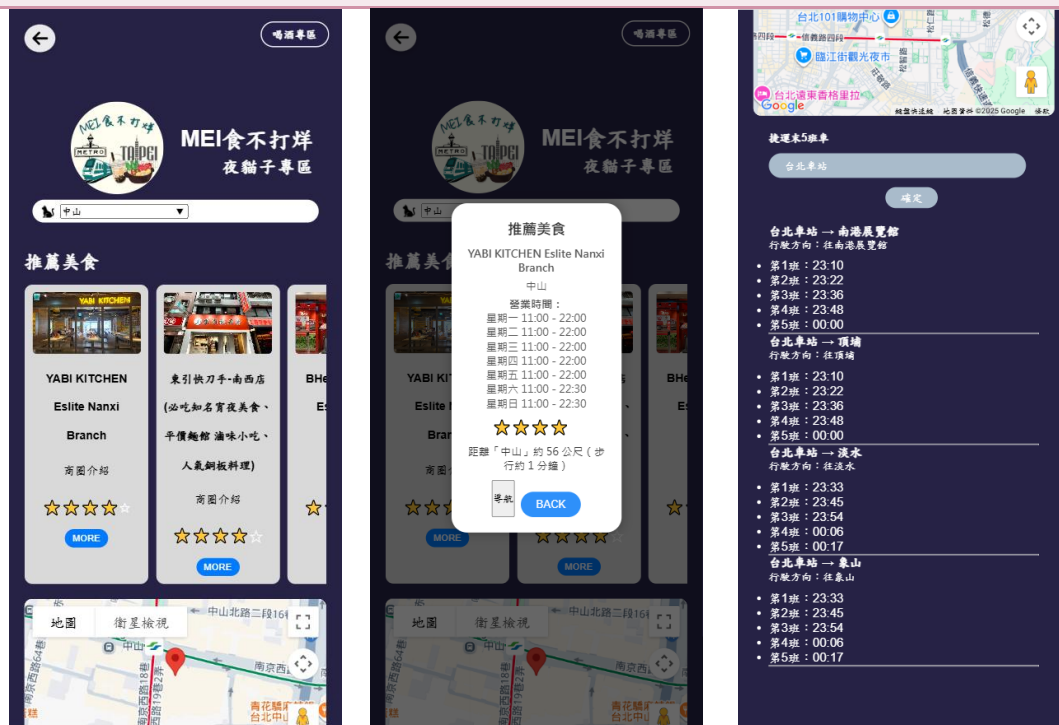
當月 TOP 10 店家



▲圖 11-1-6：當月 TOP 10 店家

使用者當不知道捷運站周邊有何美食時，我們有提供當月評論星數綜合排名前 10 名的店家資訊，使用者可透過點選來決定要去哪裡。

夜貓子專區

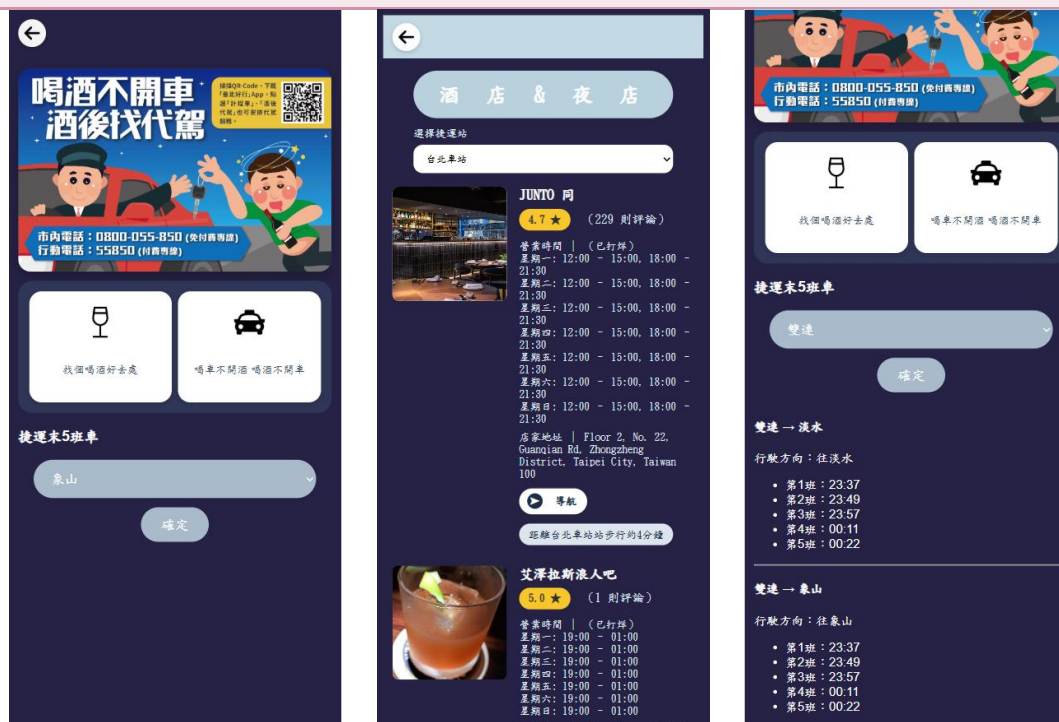


使用者可透過此專區來尋找夜間美食，除了有店家的詳細資訊外，也有捷運末五班車時刻表，幫助使用者不會錯過末班車。

a. 查詢站點周邊美食 b. 店家營業資訊 c. 末五班車資訊

▲圖 11-1-7：夜貓子專區

喝酒專區



使用者想喝酒時，可透過此專區尋找酒吧和夜店，除了有店家詳細營業資訊外，也提供代駕服務，點選圖示，可跳轉至Uber 頁面。

a. 喝酒專區 b. 酒吧&夜店營業資訊 c. 末五班車資訊

▲圖 11-1-8：喝酒專區

打卡任務

北捷美食打卡任務

歡迎參加我們的捷運美食打卡任務！這是一個探索台北捷運周邊美食的有趣活動。無論你是美食愛好者還是想要發掘新口味的朋友，這個任務都將帶你走遍城市的美味角落。透過打卡的方式，分享你的美食體驗，與其他參與者一起交流和討論，讓我們一起享受這段美食之旅！

美食任務版

打卡任務規則：

- 1. 挑選北捷任一站的店家分享美食
- 2. 需拍攝至少一張包含美食或店家的照片
- 3. 將你的照片和心得分享至我們的評論區
- 4. 完成打卡後，以便大家能夠欣賞這些美食體驗

店家名稱：

捷運站名：

推薦指數：

上傳圖片或影片： 未選擇任何檔案

評論區：

進階

北捷美食打卡任務

歡迎參加我們的捷運美食打卡任務！這是一個探索台北捷運周邊美食的有趣活動。無論你是美食愛好者還是想要發掘新口味的朋友，這個任務都將帶你走遍城市的美味角落。透過打卡的方式，分享你的美食體驗，與其他參與者一起交流和討論，讓我們一起享受這段美食之旅！

美食任務版

打卡任務規則：

- 1. 挑選北捷任一站的店家分享美食
- 2. 需拍攝至少一張包含美食或店家的照片
- 3. 將你的照片和心得分享至我們的評論區
- 4. 完成打卡後，以便大家能夠欣賞這些美食體驗

店家名稱：

捷運站名：

推薦指數：

上傳圖片或影片： 未選擇任何檔案

評論區：

✓ 打卡成功！感謝你的參與！

精選TOP10店家

北捷美食任務版

肉多多火鍋-台北西門店

☆4.7 (5則評論)

營業時間 | 11:30 - 03:30

店家地址 | 108, Taiwan, Taipei City, Wanhua District, Hanzhong St, 42號4樓

距離西門站的 260 公尺 (步行的 3 分鐘)

[導航](#) [收藏](#)

[查看詳情](#) [閱讀 \\$200-400](#) [官方網站](#)

瞞著爹西門壽司丼(海鮮丼飯)

☆4.3 (4則評論)

營業時間 | 11:30 - 21:30

店家地址 | 1樓 No. 4, Alley 2, Lane 82, Section 2, Wuchang St, Wanhua District, Taipei City, Taiwan 108

距離西門站的 384 公尺 (步行的 5 分鐘)

[導航](#) [收藏](#)

[查看詳情](#) [閱讀 —](#) [官方網站](#)

熱氣鍋物-板橋新埔店 | 寵物友善餐廳 | 板橋火鍋 | 新埔民生站火鍋 | 新埔站火鍋

☆4.6 (2則評論)

營業時間 | 11:30 - 21:30

店家地址 | No. 46號, Section 3, Minsheng Rd, Banqiao District, New Taipei City, Taiwan 220

距離新埔民生站的 124 公尺 (步行的 2 分鐘)

a. 打卡任務頁面

b. 評論美食

c. 美食任務版

使用者可參加此活動，為店家評分，也讓更多使用者看到最真實的評價。

返回

肉多多火鍋-台北西門店

☆4.7 (1294 則 Google 評論)

週二 - 週三 11:30 - 03:30

地址: 108, Taiwan, Taipei City, Wanhua District, Hanzhong St, 42號4樓

[Google 評論](#) [線上訂位](#) [分享餐廳](#)

地圖 **衛星檢視**

美食打卡任務版 | 評論區

2@gmail.com 2025/10/6 好吃扁扁扁

13@gmail.com 2025/10/5 yummy

d. 任務版店家詳細資訊

美食打卡任務版 | 評論區

2@gmail.com 2025/10/6 好吃扁扁扁

13@gmail.com 2025/10/5 yummy

45 2025/6/3 好超好吃

45 2025/4/22 多

45 2025/4/22 林慶威覺得好吃

e. 打卡評論內容

▲圖 11-1-9：打卡任務

美食地圖



a. 美食地圖頁面



b. 查看 Google 評論

使用者可使用捷運站名稱來搜尋站點周邊的美食資訊，從[a]的紅框點進去可看到[b]Google 評論及地圖。

▲圖 11-1-10：美食地圖

收藏及通知



a. 美食任務版



b. 美食地圖



c. 通知

使用者可在【美食任務版】、【美食地圖】收藏美食，收藏後的美食會跳通知在通知頁面，可將訊息判斷為未讀或已讀。

▲圖 11-1-11：收藏及通知

會員中心



a.會員中心頁面

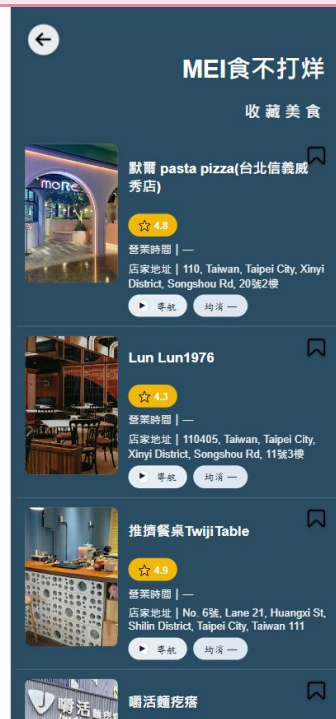


b.修改個人資料

使用者可修改會員資料，也可更換頭貼照片，修改完成後按下儲存即可。

▲圖 11-1-12：會員中心

收藏美食



▲圖 11-1-13：收藏美食

使用者收藏的美食都會出現在【收藏美食】頁面，裡面有店家詳細資訊，也可在此取消收藏。

第 13 章 感想

 11146061 郭宥妍

在本次專題之前，我曾在高中參與過類似大學專題的模式，當時同樣擔任組長，負責統籌並帶領團隊進行系統的開發與製作，整體規模皆比照大學專題進行。這段經驗讓我在面對此次專題時更加從容，自信地面對各種流程與挑戰，不僅不會感到緊張，也能協助組員更快熟悉進度，讓整體製作過程更加順利與高效。與以往不同的是，這次我首次接觸 UI/UX 設計。在剛開始使用 Figma 進行介面設計時，由於系統介面全為英文，加上毫無使用經驗，讓我一開始完全不知從何著手。為了克服這個障礙，我積極查閱教學資源與範例，並透過不斷的練習與嘗試，逐漸熟悉操作方式，也對設計的流程與原則有了初步的理解，這是我過去未曾接觸的領域，帶給我全新的挑戰與收穫。

此外，我在團隊領導與溝通協調方面也有明顯成長。我學會了如何更有效地分配工作內容，妥善安排會議時間，並在與老師討論時清楚表達進度與問題，同時也能處理各種突發的瑣碎事項。這次專題不僅提升了我的技術與設計能力，也讓我在組織、溝通與領導方面更加成熟，為未來面對更大專案奠定了穩固的基礎。

 11146066 簡孝宇

這次專題從「把資料串起來」開始，一路走到能用的服務，心情像坐捷運轉好幾次車。最難的是資料對不起來、連線有時失敗、手機畫面又常走鐘；常常改好一處，另一處又壞。後來把目標拆小：先讓基本查詢能用，再慢慢補照片、路線、導航。遇到怪問題就先記錄步驟、重現一次、用最簡單的方法先通過，之後再美化。過程中學到耐心與溝通的重要：誰負責哪一塊、改了要說清楚、寫下備忘與說明。雖然不完美，但看見頁面一步步成形，很有成就感；之後也想補更多提示、提升速度，讓整體更順更友善。

 11146073 張楷偉

在這次的專題製作過程中，我學到了許多課堂上接觸不到的實作經驗。從一開始構思主題、規劃架構，到實際撰寫前端畫面與資料呈現的過程中，一路上也遇到很多的難關，例如在設計網頁版面時，要兼顧排版、美觀與響應式設計（RWD），常常一個小細節就得反覆修改；在處理資料與互動功能時，也要考慮使用者體驗與操作流程，這些都讓我更加了解網頁開發的細節與難處。

過程中也經歷過不少挫折，像是版面跑版、功能不如預期、檔案路徑錯誤等問題，常常需要花許多時間除錯。不過透過不斷嘗試、查資料與請教同學，逐漸把問題一一解決。看到網頁從草稿一步步變成可實際操作的成品，心中充滿成就感。這次專題不僅讓我提升了網頁設計的能力，也讓我學會面對困難時保持耐心、解決問題的態度，這將成為我未來學習與成長的重要養分。

本次專題負責資料庫、後端、虛擬機的整合，在資料庫的部分，前期建置商家資料有遇到許多問題，像是要如何完整的取出商家的留言，原本想說使用 google place api 就可以將留言都抓下，後來發現是可以抓留言，但一次只有五筆的資料，後來將爬蟲技能用上來，才將大筆大筆的資料抓下，最後帶入到資料庫當中，後端在建置 api 時也有許多的困難，要先預想前端會需要那些資料，怎麼樣把資料傳遞會速度較快，當前端接收有問題時要幫忙解決哪裡出現了 bug，虛擬機的部份為了要一直 update 餐廳頻論，所以有做了一些檔案讓虛擬機也能更新資料，本地端移到虛擬機上又會是新的考驗。

第 14 章 參考資料

- [1] ChatGPT
- [2] UI/UX Figma <https://www.youtube.com/@CreativeKev90>
- [3] 餐廳評論爬蟲 <https://ithelp.ithome.com.tw/m/articles/10267131>
- [4] 圖片存放位置 <https://ithelp.ithome.com.tw/articles/10330627?sc=pt>
- [5] 北捷 api <https://www.metro.taipei/cp.aspx?n=BDEB860F2BE3E249>
- [6] Django <https://ithelp.ithome.com.tw/articles/10206355>
- [7] 批次檔定期更新 <https://ithelp.ithome.com.tw/articles/10369742>
- [8] Django 資料庫 <https://ithelp.ithome.com.tw/m/articles/10301639>

附錄

評審問題回覆

評審建議事項	修正情形
宜補上定時撈取餐廳資料的介紹。	我們有在虛擬機設置一天一次抓取餐廳的資料，並且在每天凌晨 3 點都會更新。

評審建議事項	修正情形
檢索排序納入餐廳打烊時間、走路距離、人潮、留言的考量。	我們有新增餐廳從幾點營業到打烊時間，以及利用 Google map 計算從捷運站步行至餐廳的時間，排序方式為步行時間近到遠排序。

評審建議事項	修正情形
「美食打卡」可導入 Google 評論資料。	我們團隊目前技術尚未能達到此技術，且另一方面擔心導入 Google 評論後，會將我們原本系統上的評論給刷掉，因此我們採用外接方式導到 Google 評論。

評審建議事項	修正情形
結合代駕服務、深夜便利商店及喝酒專區，強化夜貓子功能。	我們有新增「喝酒專區」，此專區從夜貓子專區進入，喝酒專區內有酒吧和夜店的店家資訊及代駕專區，代駕專區我們直接連結到 Uber 的頁面，另外也有加上捷運末五班車的資訊在喝酒專區，可讓用戶不需再跳轉到上一頁搜尋捷運末班車。

評審建議事項	修正情形
站點搜尋不要用下拉式選單，可以用地圖呈現比較直觀。	我們考量到製作時間且下拉式選單較比直接地圖點選方式更直觀，所以我們決定維持原樣使用下拉式選單來搜尋站點。

評審建議事項	修正情形
未來可建議讓民眾導入行程，提供交通資訊和推薦美食功能，可參考 MaaS 融合食宿遊購行概念。	我們接受此建議，但目前尚未能提供這項服務，我們預期在複評後可以增進這項服務。

評審建議事項	修正情形
店家營業時間篩選，宜呈現不打烊店家的資訊	已新增篩選功能，已打烊的店家會顯示在頁面最底下，正在營業的會顯示在上面。

評審建議事項	修正情形
整合 Google 評論，並非使用自身資料庫	新增 Google 評論至頁面上，可隨意切換資料庫。